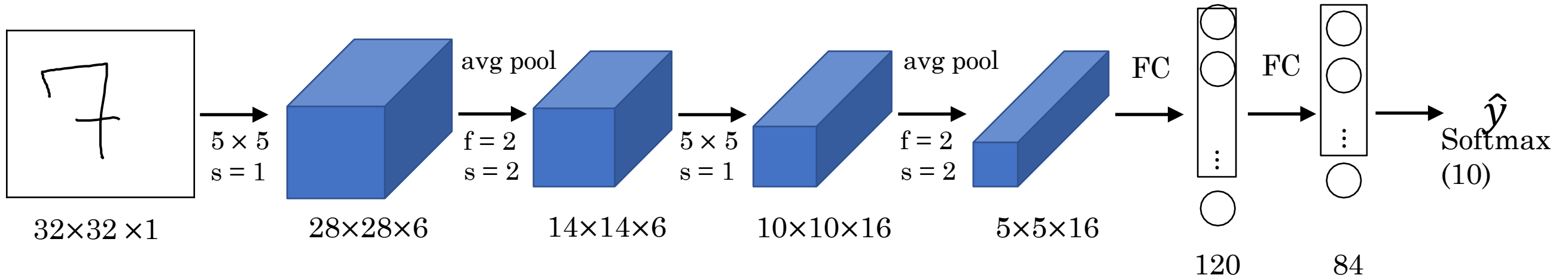

CNN Архитектуры

Основни архитектуры

[LeNet] → [AlexNet] → [VGG] → [ResNet] → [Inception] → [MobileNet]

LeNet - 5



60 000 параметри

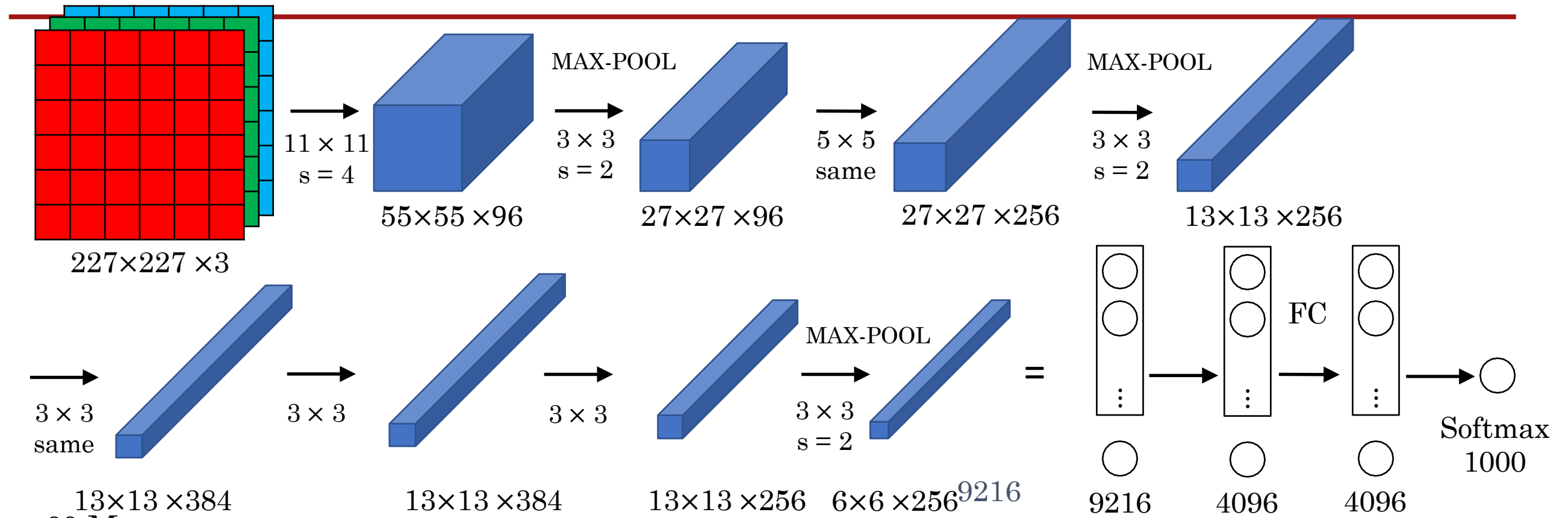
- Няма допълване
- Активация сигмоид и хиперболичен тангенс

[LeCun et al., 1998. Gradient-based learning applied to document recognition]

Архитектури

- ✓ **LeNet** . Първите успешни приложения на конволюционните мрежи са разработени от Yann LeCun през 1990 г. От тях най-известната е архитектурата LeNet , която се използва за четене на пощенски кодове, цифри и т.н.
- ✓ **AlexNet** . Първата работа, която популяризира конволюционните мрежи в компютърното зрение, е AlexNet , разработена от Алекс Крижевски, Иля Сутскевер и Джеф Хинтън. AlexNet е представен на предизвикателството ImageNet ILSVRC през 2012 г. и значително надминава втория подгласник (топ 5 грешки от 16% в сравнение с подгласника с 26% грешка). Мрежата е много подобна архитектура на LeNet, но е по-дълбока, по-голяма и включва конволюционни слоеве, подредени един върху друг (преди това е обичайно да има само един слой CONV, последван веднага от слой POOL).

AlexNet



- 13x13x384
- 60 М параметри
- ReLU
- Няколко GPU
- Local Response Normalization
- С допълване

[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Historical note: Trained on GTX 580 GPU with only 3 GB of memory.
Network spread across 2 GPUs, half the neurons (feature maps) on each GPU.

Full (simplified) AlexNet

architecture: [227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

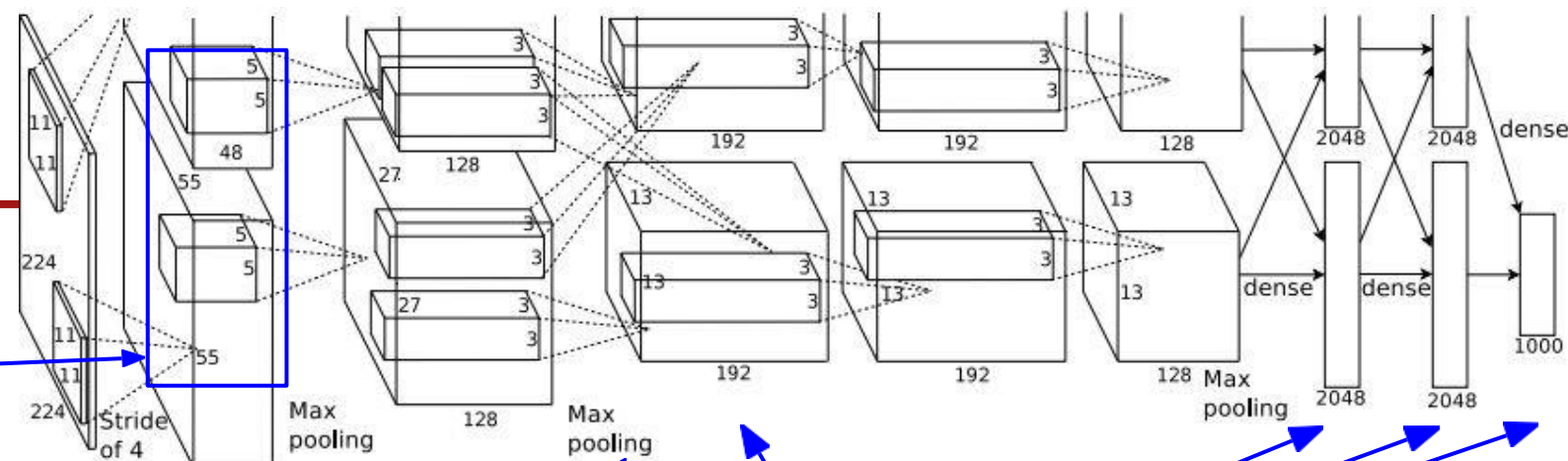
[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

[55x55x48] x 2



CONV1, CONV2, CONV4, CONV5:
Connections only with feature maps on same GPU

CONV3, FC6, FC7, FC8:
Connections with all feature maps in preceding layer, communication across GPUs

AlexNet

[Krizhevsky et al. 2012]

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

VGG

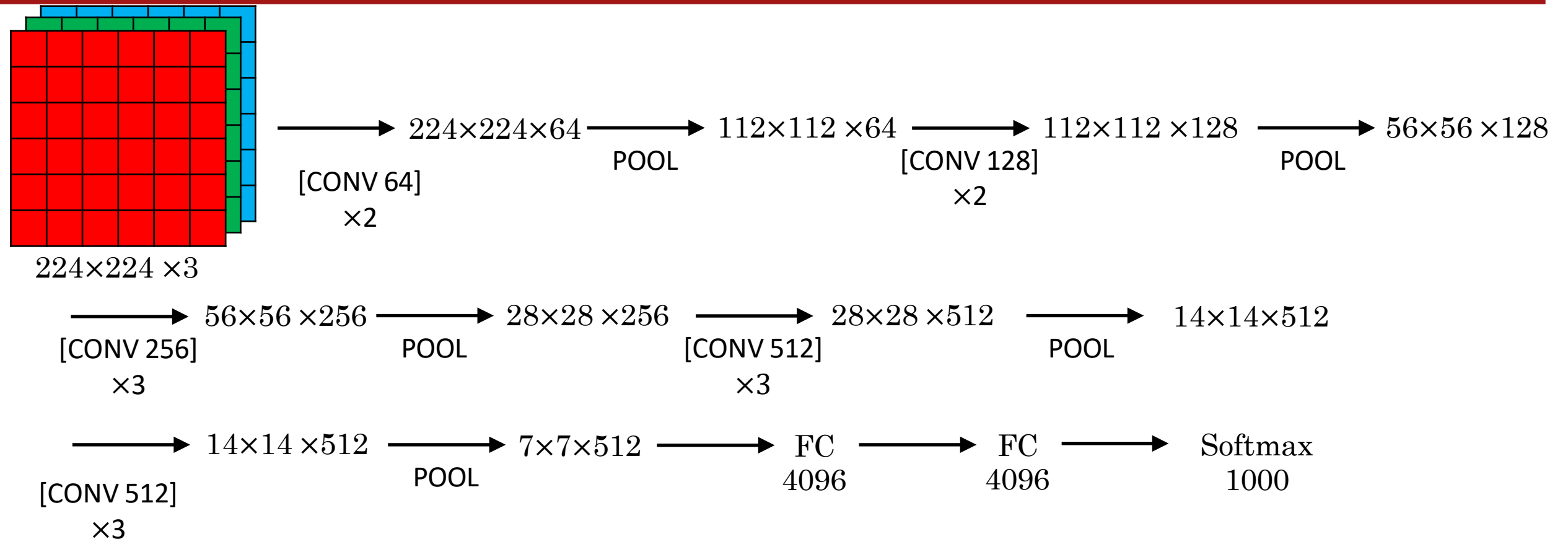
- **VGGNet** . Подгласник в ILSVRC 2014 е мрежата на Карен Симонян и Андрю Зисерман, която е известна като VGGNet . Неговият основен принос е да покаже, че дълбочината на мрежата е критичен компонент за добра производителност. Отличава се с хомогенна архитектура, която изпълнява само **3x3 конволюции и 2x2 обединяване** от началото до края. Техният предварително обучен модел е достъпен за използване. Недостатък на VGGNet е, че е по-скъп за оценка и използва много повече памет и параметри (140M). Повечето от тези параметри са в първия напълно свързан слой и оттогава е установено, че тези FC слоеве могат да бъдат премахнати без понижаване на производителността, което значително намалява броя на необходимите параметри (1x1 conv)

VGG - 16

CONV = 3×3 filter, $s = 1$, same

VGG - 19

MAX-POOL = 2×2 , $s = 2$

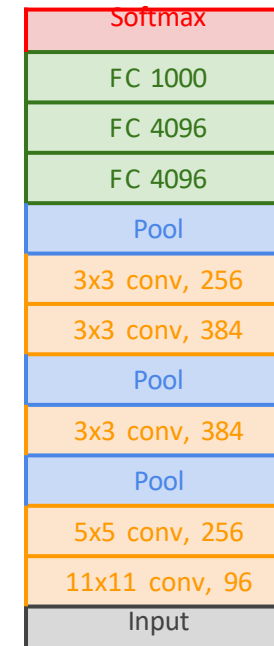


- ✓ ~ 138 M параметри
- ✓ ReLU
- ✓ conv->conv

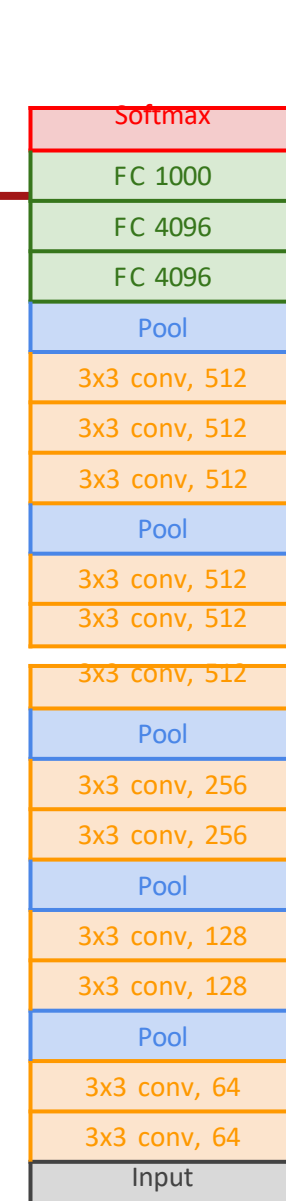
[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

VGGNet

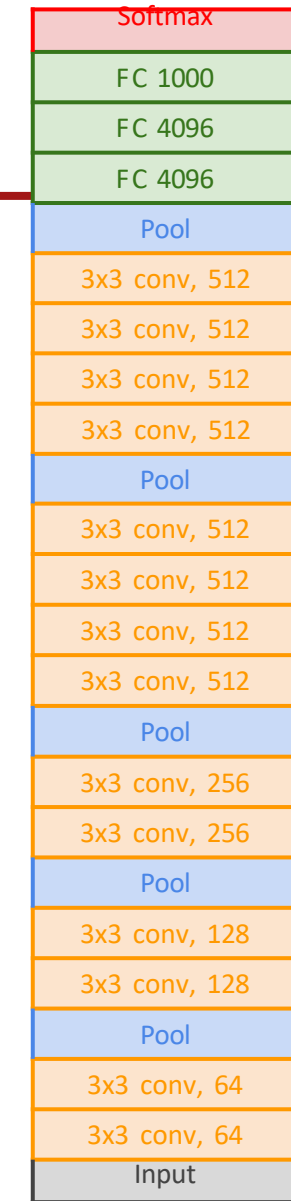
- Q: Защо да се използват по-малки филтри? (3x3 конв)
- Стек от три 3x3 конв (стъпка 1) слоя има същия
- Ефективно рецептивно поле като един 7x7 конв слой
- Но по-дълбоки, по-нелинейни
- И по-малко параметри: $3 * (3^2 C^2)$
- срещу $7^2 C^2$ за C канали на слой



AlexNet



VGG16



VGG19

[Simonyan and Zisserman, 2014]

- INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0
- CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$
- CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$
- POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0
- CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$
- CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$
- POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
- CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
- POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
- CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
- POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0
- CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
- CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
- CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
- POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0
- FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$
- FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$
- FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$
- TOTAL memory: 24M * 4 bytes $\sim$ 96MB / image (only forward! $\sim$ *2 for bwd)

Повечето памет е в
ранните слоеве
връзка (CONV)

Most params are in late FC

TOTAL params: 138M parameters

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

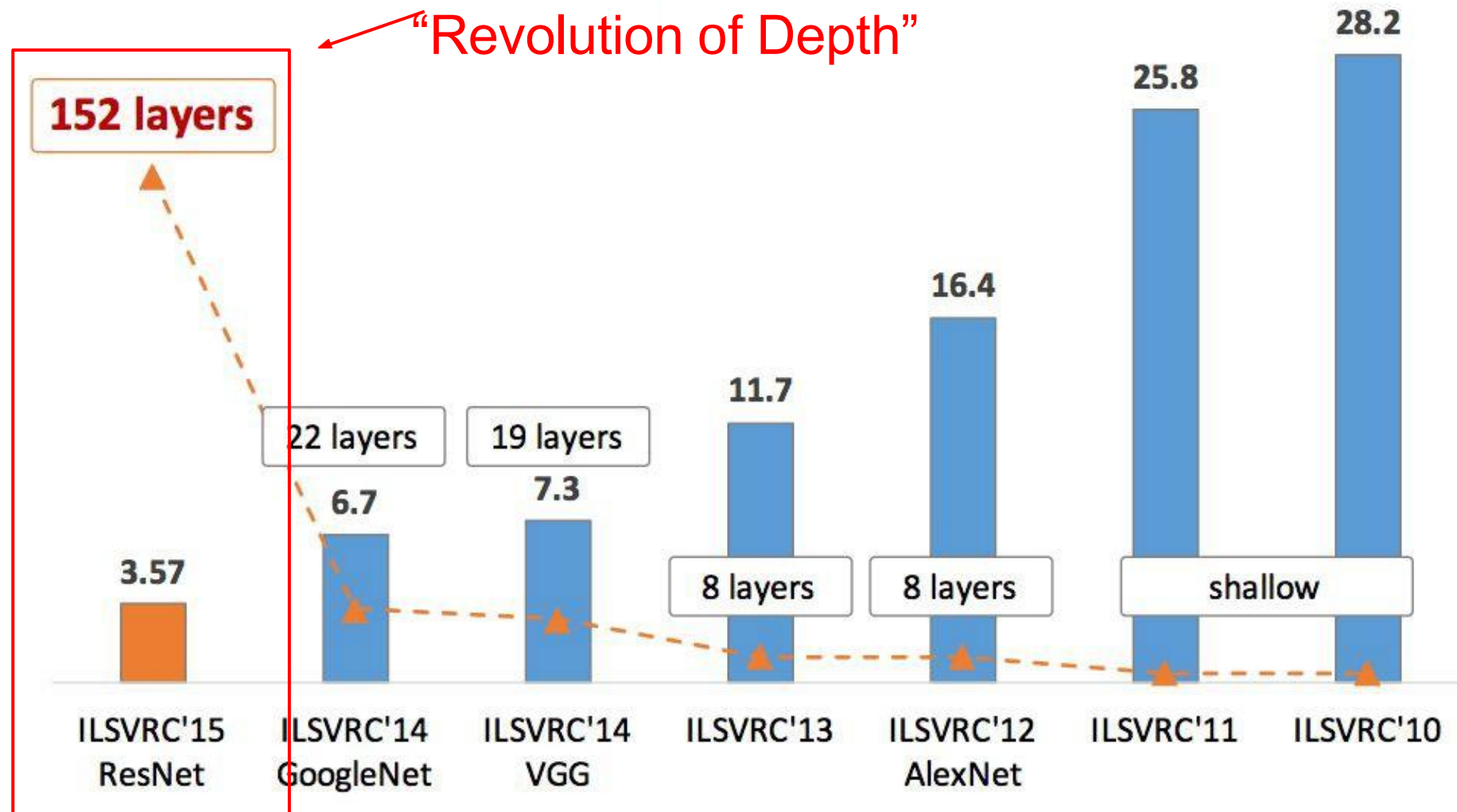


Figure copyright Kaiming He, 2016. Reproduced with permission.

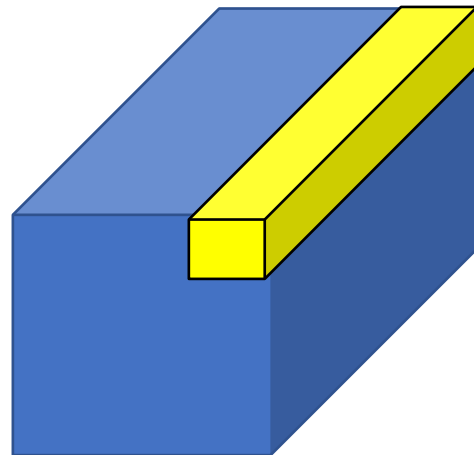
Мрежа в мрежата

1×1 КОНВОЛЮЦИЯ

1 × 1 КОНВОЛЮЦИЯ

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 1



6 × 6 × 32

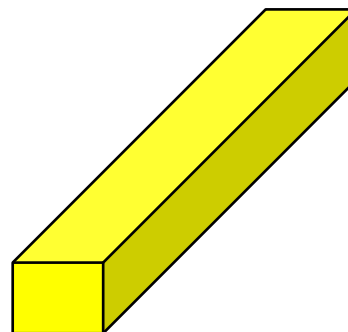
*

2

=

2	4				

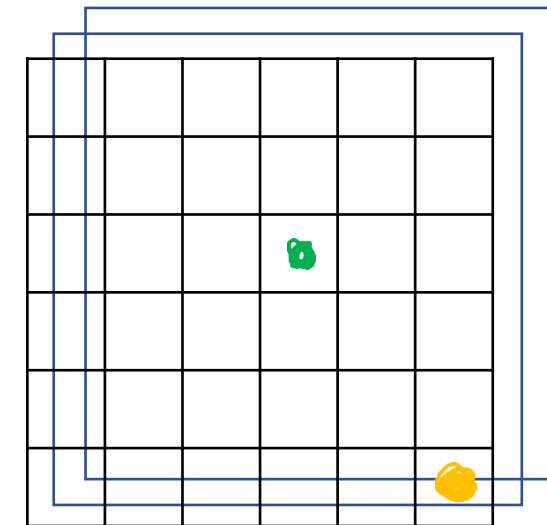
*



1 × 1 × 32

=

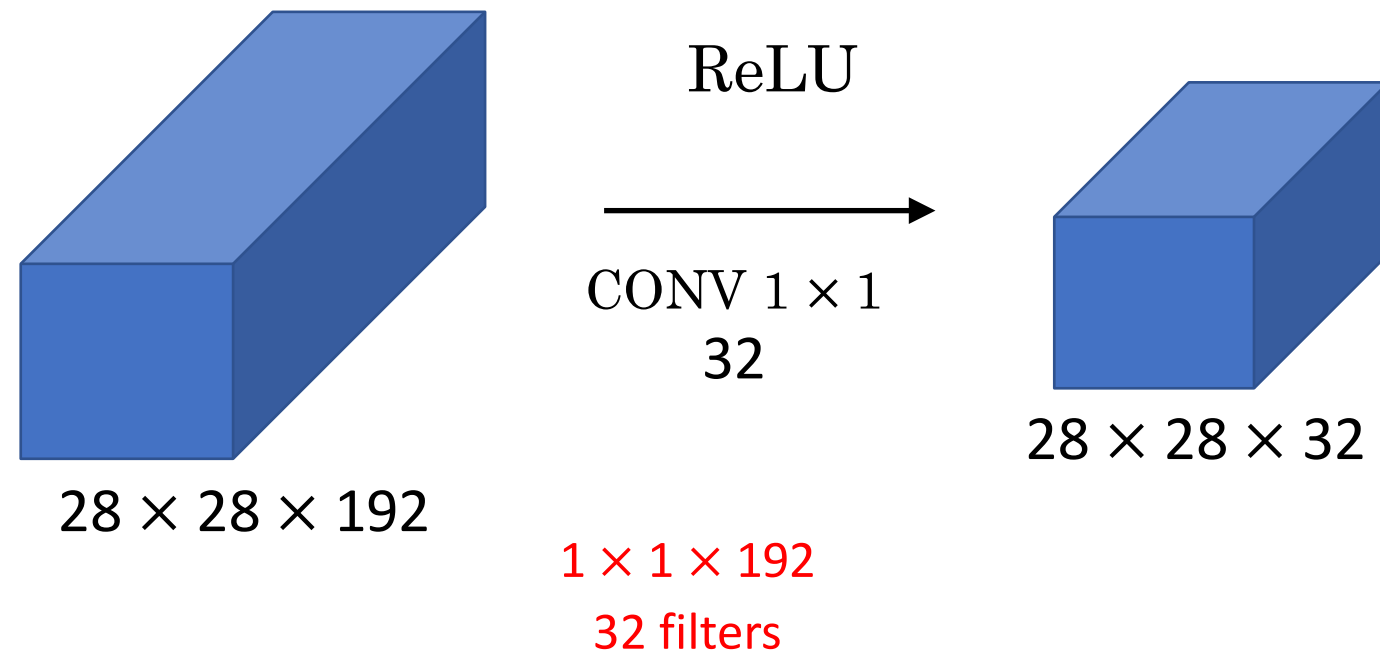
ReLu
Network in
Network



6 × 6 × # filters

[Lin et al., 2013. Network in network]

1×1 КОНВОЛЮЦИЯ

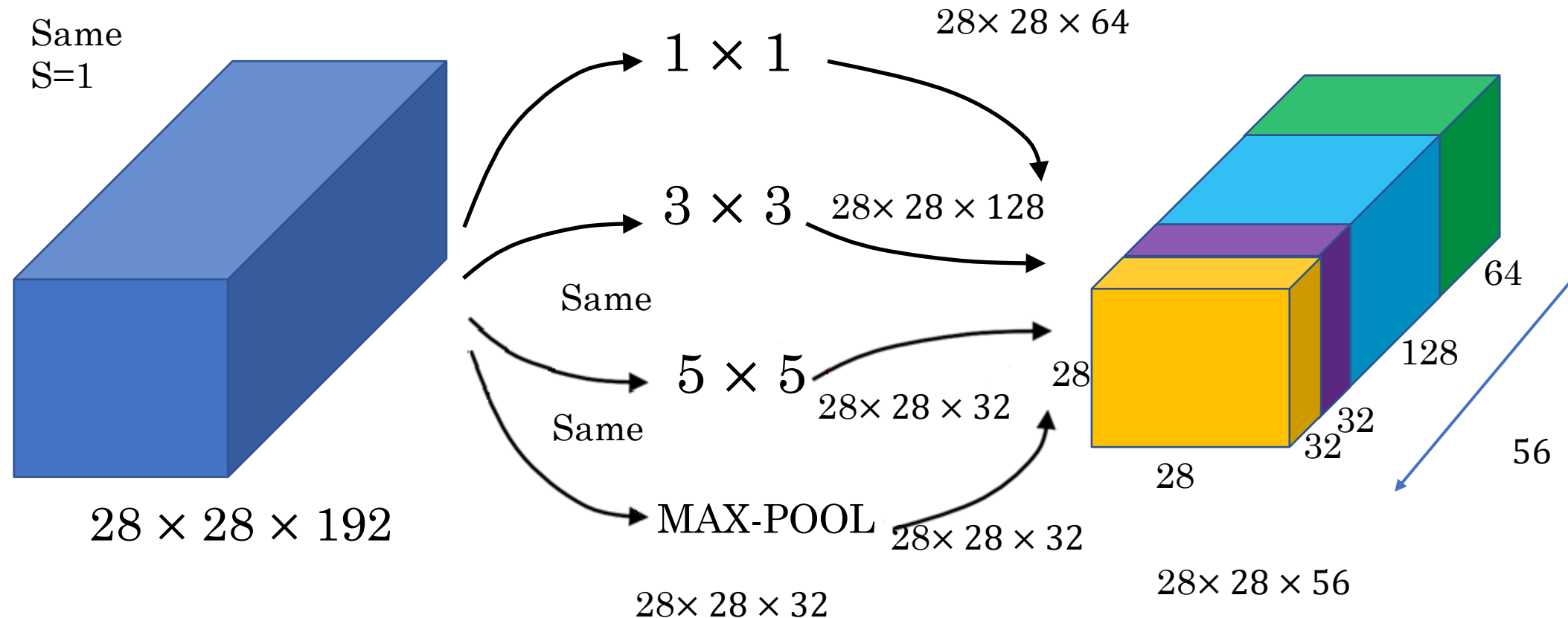


- 5×5 конволюции са изчислително скъпи, което мотивира използването на 1×1 конволюция за намаляване на размерността, преди прилагане на по-големи филтри

Inception network



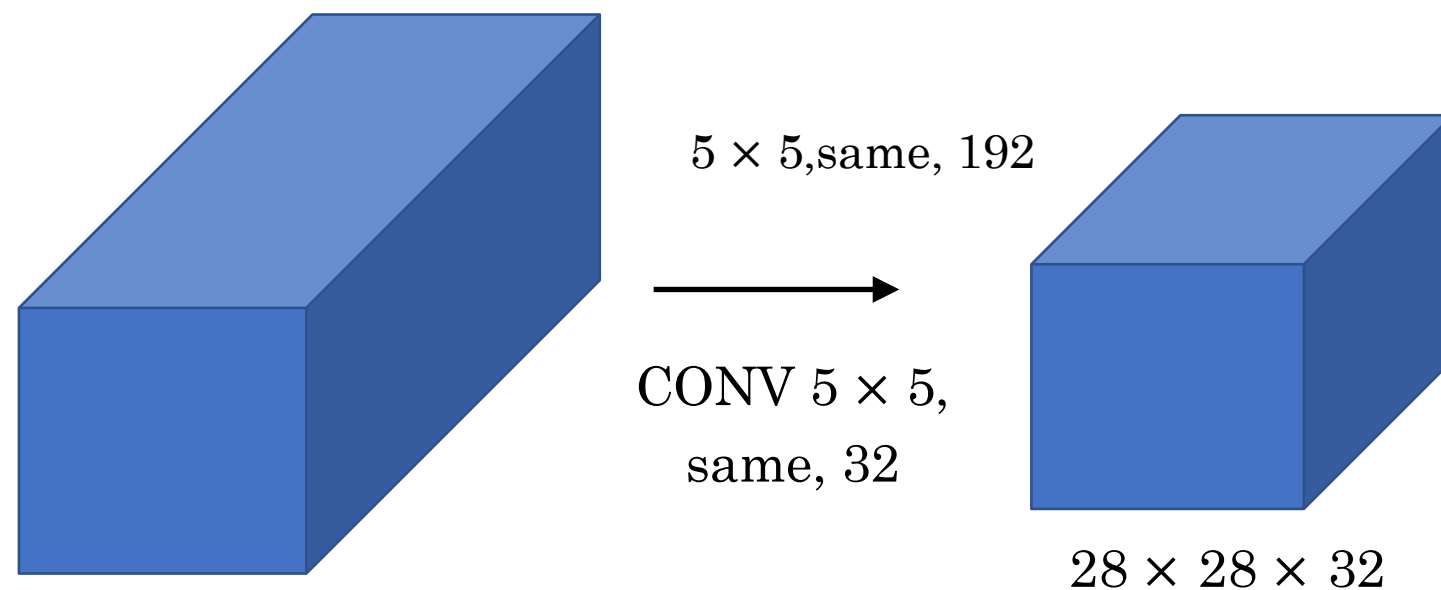
inception network



- Inception модулите обработват един и същ вход с множество размери на филтри паралелно и конкатенират резултатите, позволявайки на мрежата ефективно да усвоява мултимасщабни функции

[Szegedy et al. 2014. Going deeper with convolutions]

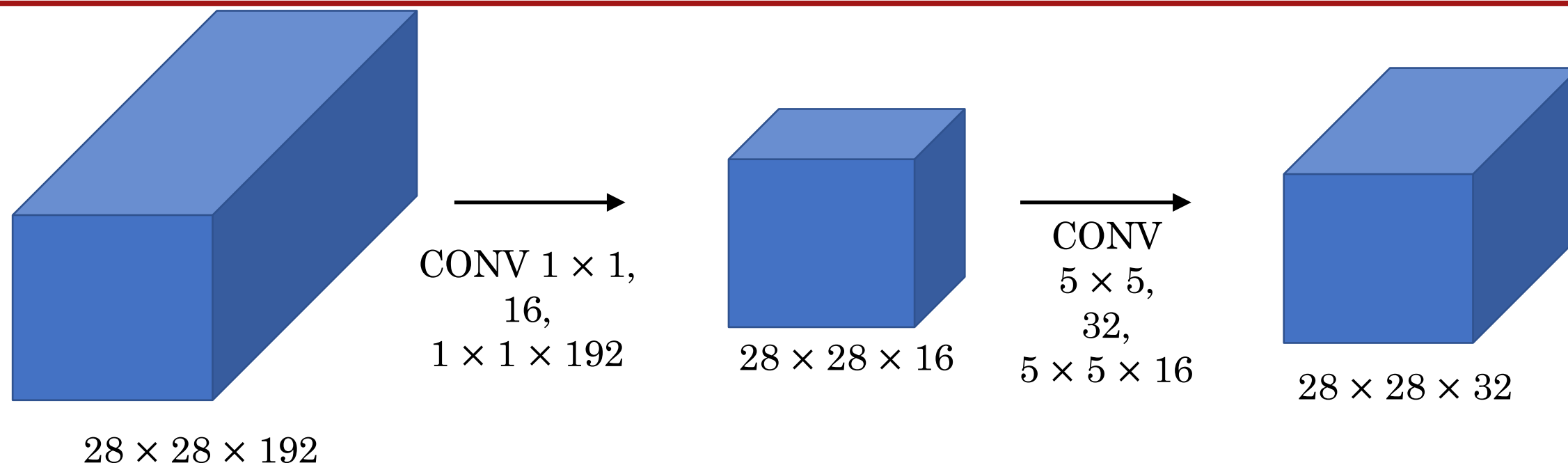
Високи изчислителните разходи



28 × 28 × 192

$$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120 \text{ М операции}$$

Конволюция 1×1 за намаляване на операциите



$$28 \times 28 \times 16 \times 192 = 2.4 \text{ M}$$

$$28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10 \text{ M}$$

1×1 convolution намалява броя канали, като комбинира информацията от тях чрез претеглена сума. Така се получава компактно представяне, което запазва най-важните характеристики. Въпреки че се губи част от информацията, мрежата се обучава да съхрани полезното за задачата. Това позволява значително намаляване на изчисленията преди по-скъпи операции (напр. 5×5 convolution).

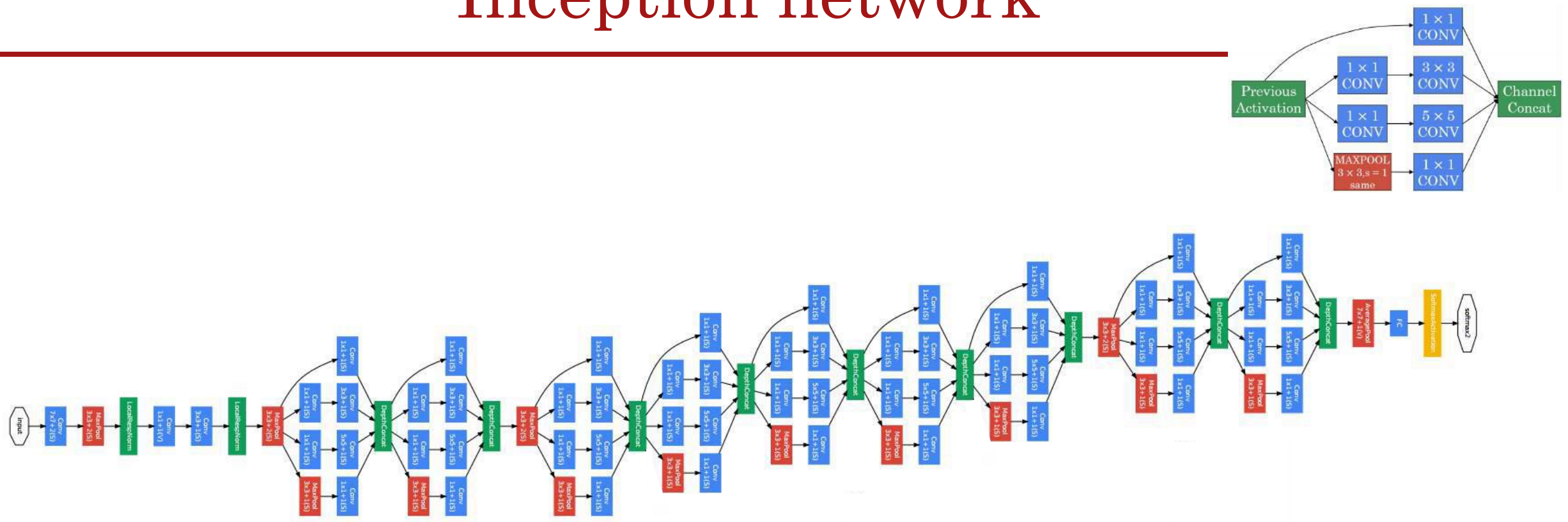
Inception network



GoogLeNet

- **GoogLeNet** . Победителят в ILSVRC 2014 е конволюционна мрежа от [Szegedy et al.](#) от Google. Основният му принос е разработването на *модул* , който драстично намалява броя на параметрите в мрежата
- (4M, в сравнение с AlexNet с 60M).
- Използва avg pooling вместо напълно свързани слоеве в горната част на ConvNet, елиминирайки голямо количество параметри, които изглежда нямат голямо значение. Има и няколко последващи версии на GoogLeNet [Inception-v4](#)

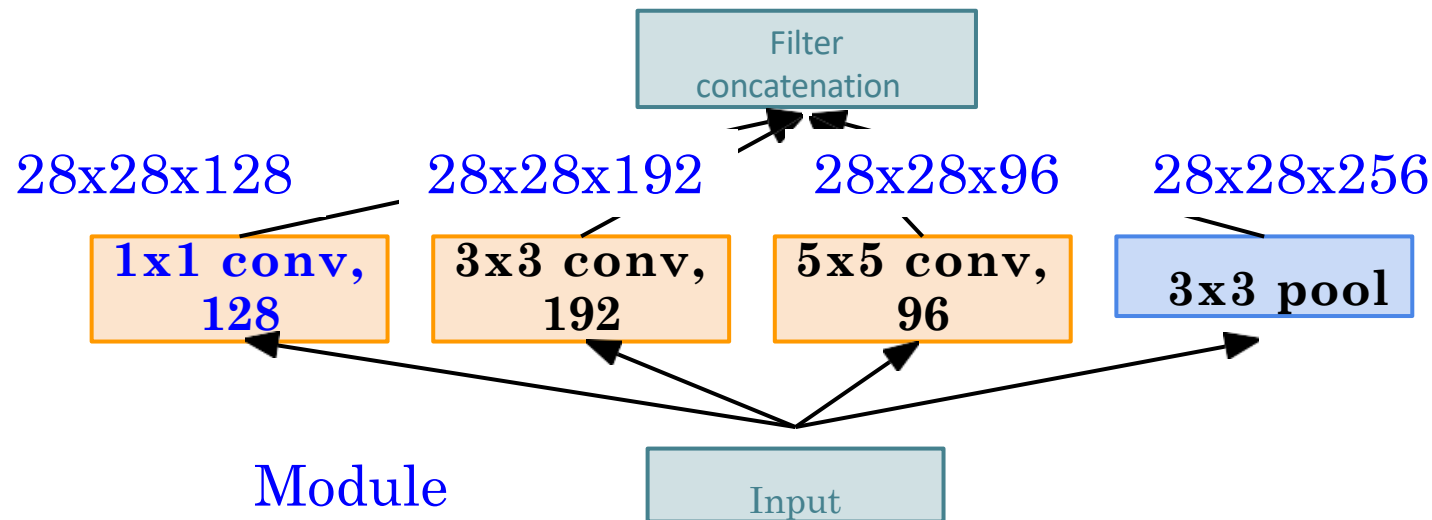
Inception network



[Szegedy et al., 2014, Going Deeper with Convolutions]

GoogLeNet

$$28 \times 28 \times (128 + 192 + 96 + 256) = 28 \times 28 \times 672$$



Module
input:

28x28x256

Naive Inception module

Conv Ops:

[1x1 conv, 128] $28 \times 28 \times 128 \times 1 \times 1 \times 256$

[3x3 conv, 192] $28 \times 28 \times 192 \times 3 \times 3 \times 256$

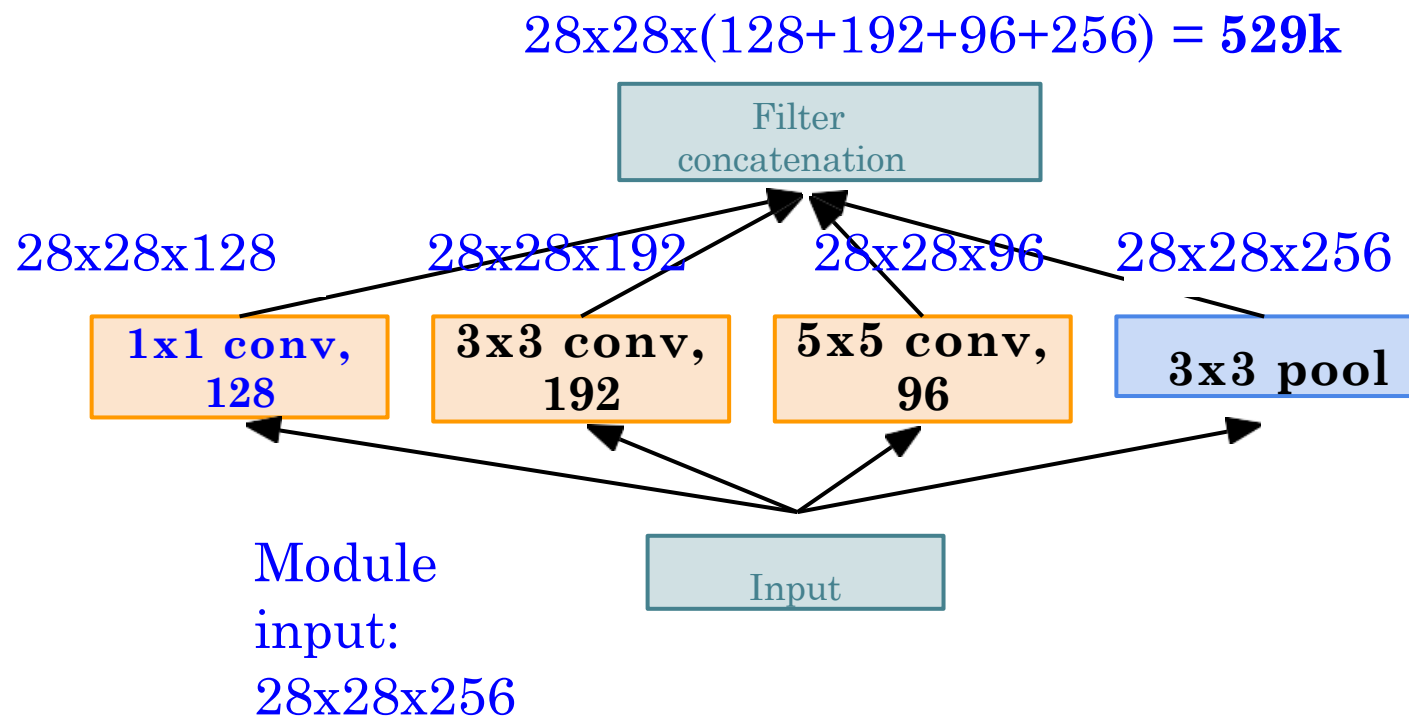
[5x5 conv, 96] $28 \times 28 \times 96 \times 5 \times 5 \times 256$

Total: 854M ops

Много скъпи изчисления

[Szegedy et al., 2014]

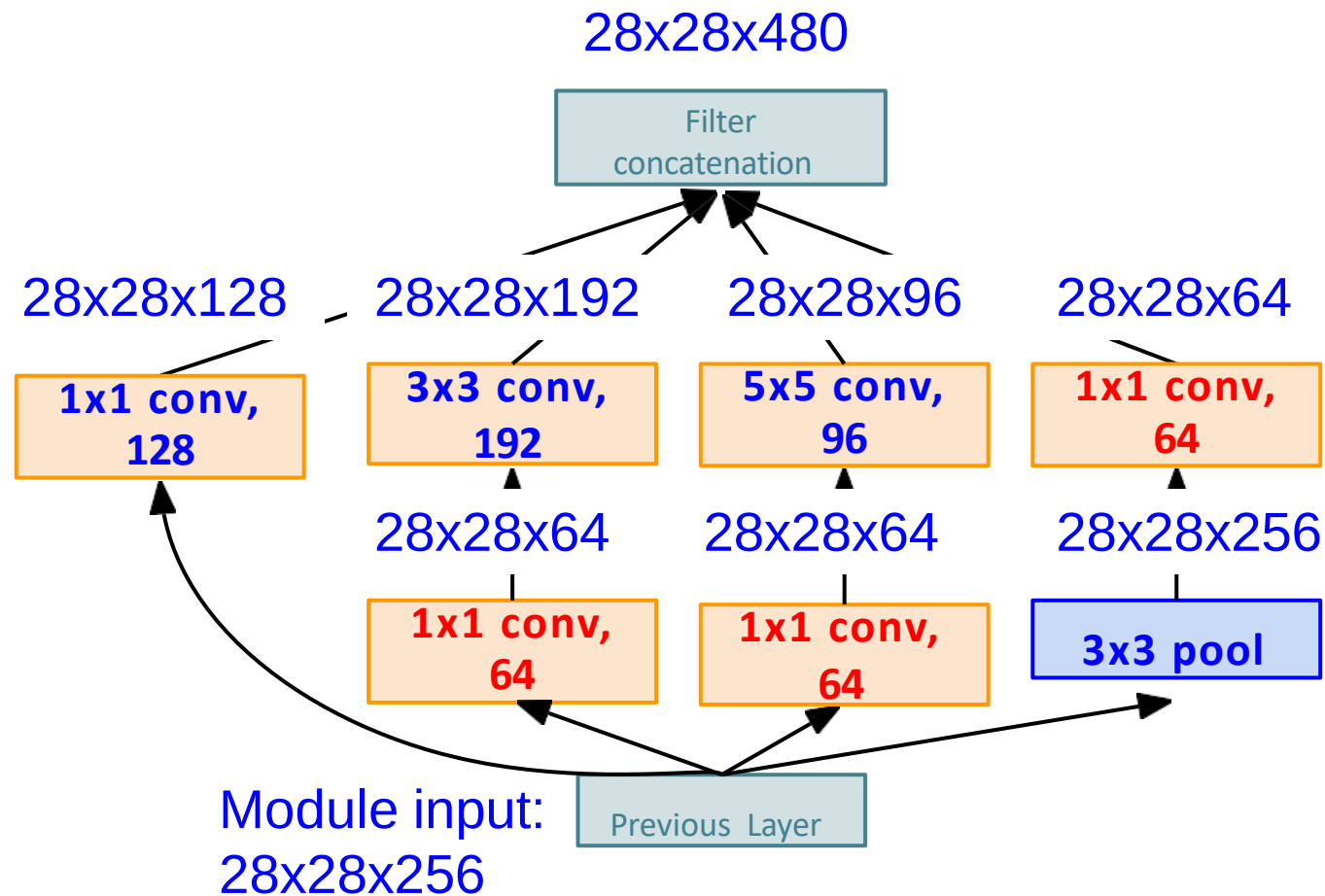
GoogLeNet



Решение: слоеве със стестняване, които използват 1x1 конволюции за намаляване на дълбочината на елементите

[Szegedy et al., 2014]

GoogLeNet



Конверсионни операции:

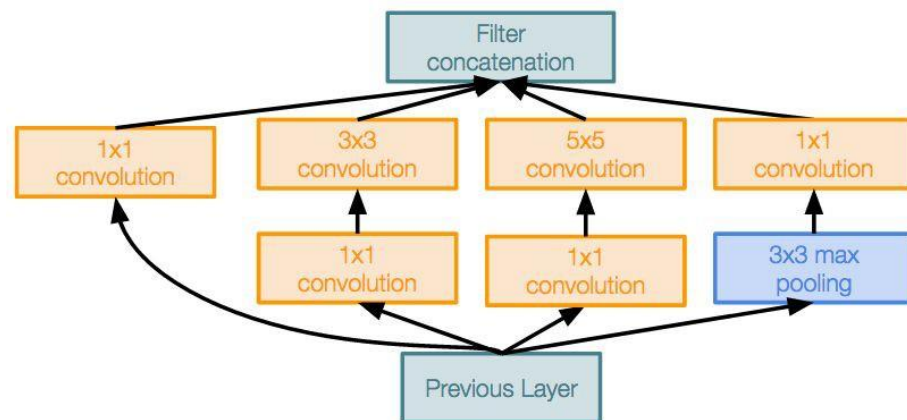
[1x1 conv, 64] 28x28x64x1x1x256
[1x1 conv, 64] 28x28x64x1x1x256
[1x1 conv, 128] 28x28x128x1x1x256
[3x3 conv, 192] 28x28x192x3x3x64
[5x5 conv, 96] 28x28x96x5x5x64
[1x1 conv, 64] 28x28x64x1x1x256
Общо: 358М операции

В сравнение с 854М операции за наивна версия, Bottleneck може да намали дълбочината след pooling layer

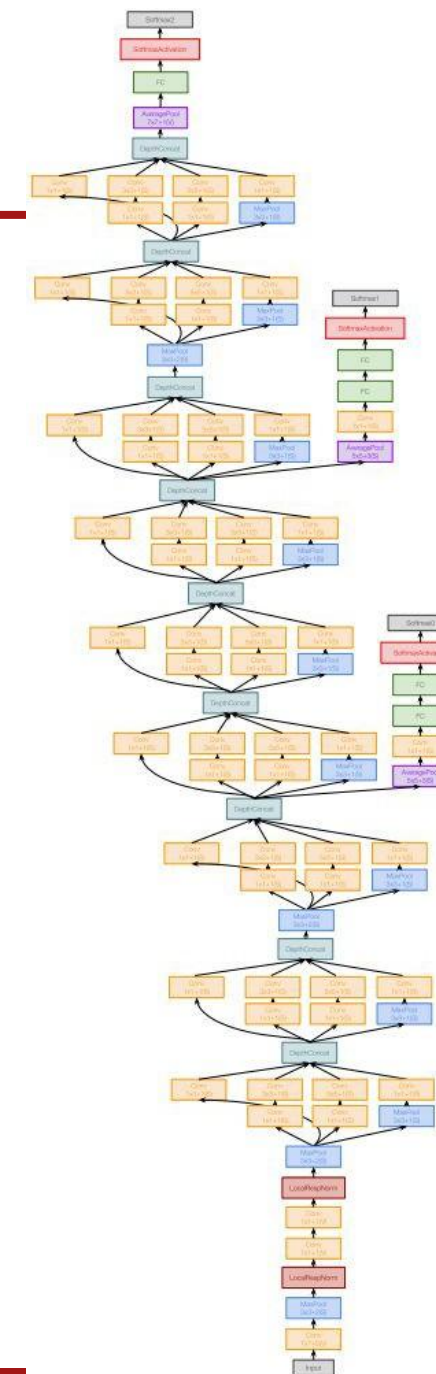
[Szegedy et al., 2014]

GoogLeNet

- По-дълбоки мрежи с изчислителна ефективност
- 22 слоя
- Без FC слоеве
- Ефективен модул "Inception"
- 12 пъти по-малко параметри от AlexNet
- Победител в класирането ILSVRC'14 (6.7% грешка в топ 5)



Inception module



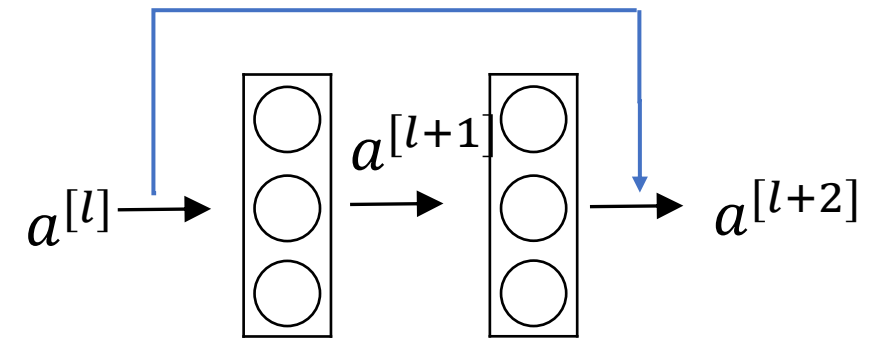
[Szegedy et al., 2014]

Остатъчни мрежи (ResNets)

Resnet

- **ResNet** . Остатъчна мрежа , разработена от Kaiming He et al. е победител в ILSVRC 2015. Включва специални *прескачащи връзки* и интензивно използване на нормализация на партии . В архитектурата също така липсват напълно свързани слоеве в края на мрежата. ResNets са изборът по подразбиране за използване на конволюционна невронна на практика.
- Други разработки, които променят оригиналната архитектура от Kaiming He et al. Съпоставяне на идентичността в дълбоки остатъчни мрежи

Residual block



Short cut/ skipped connection

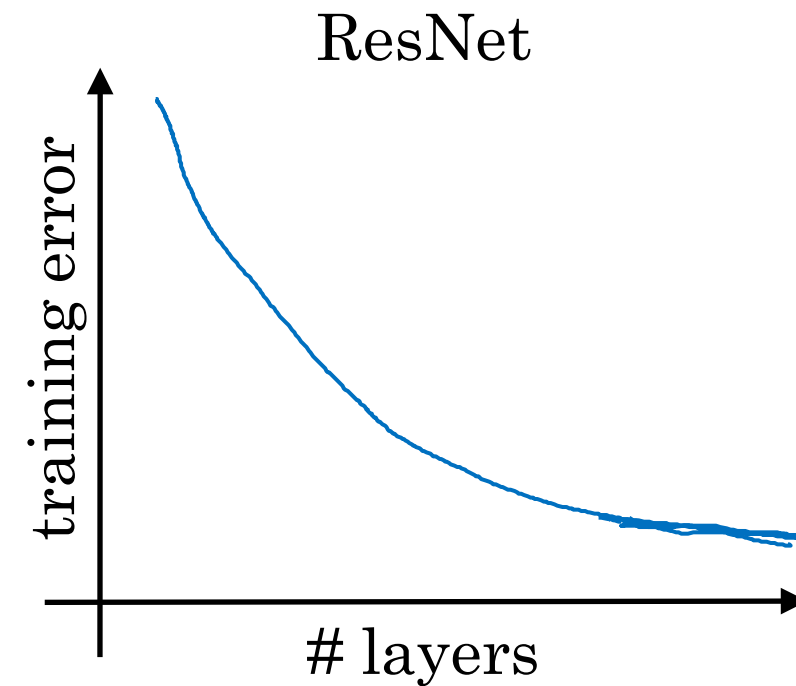
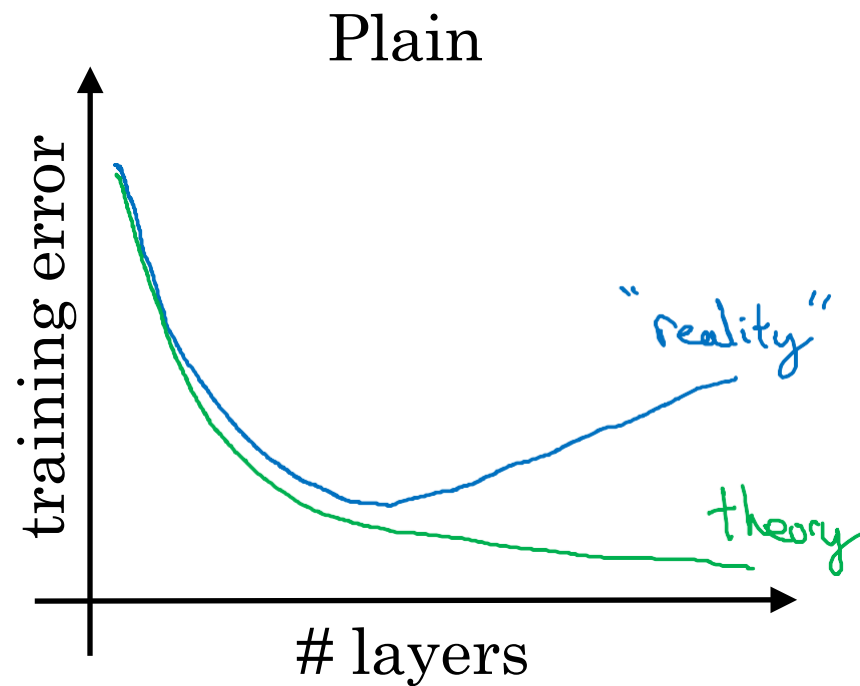
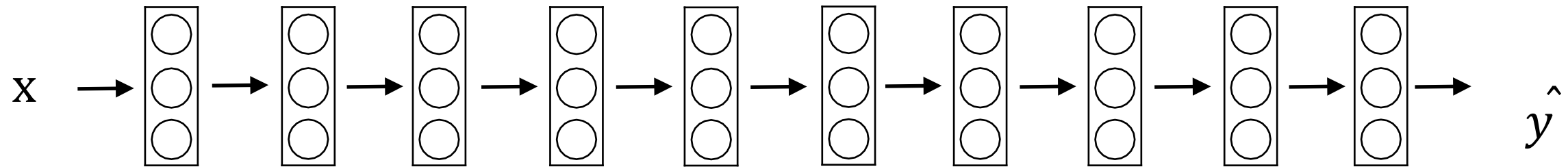
$a^{[l]} \text{ -----} \rightarrow \text{Linear} \text{ -----} \rightarrow \text{ReLU} \text{ -----} \rightarrow \text{Linear} \text{ -----} \rightarrow \text{ReLU} \text{ ----} \rightarrow a^{[l+2]}$
 $a^{[l+1]}$

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]} \quad a^{[l+1]} = g(z^{[l+1]}) \quad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

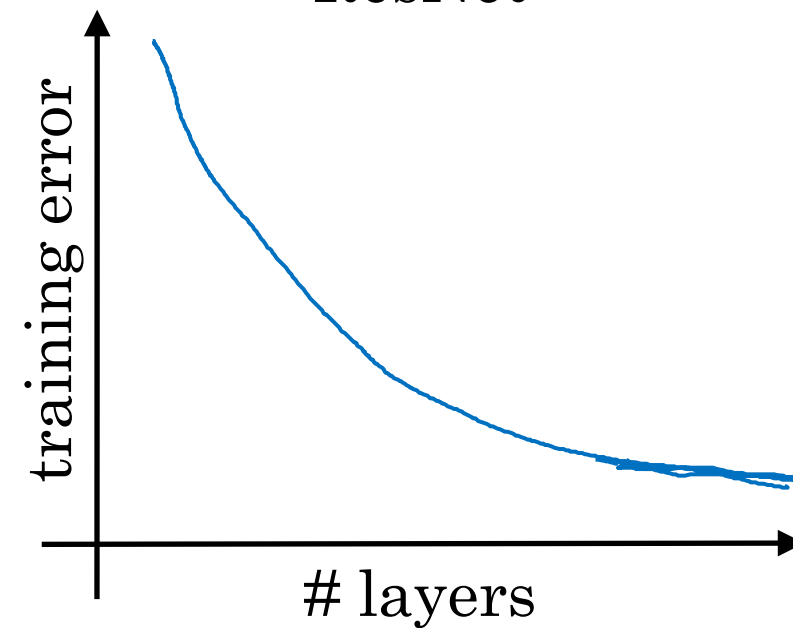
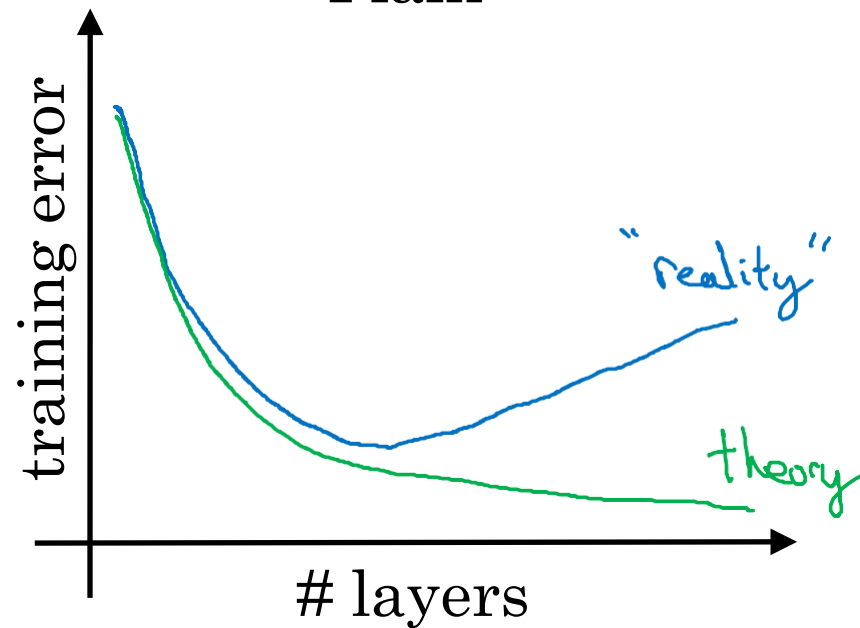
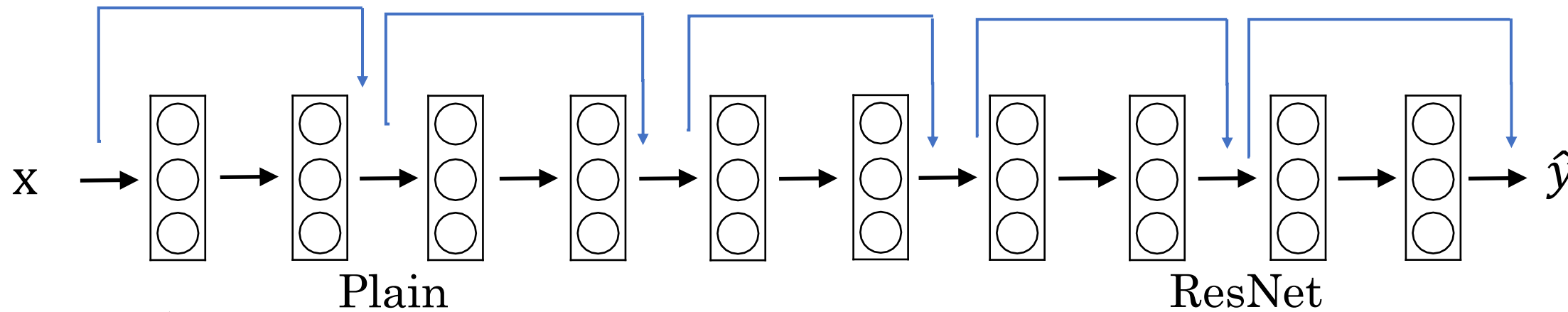
[He et al., 2015. Deep residual networks for image recognition]

Residual Network



[He et al., 2015. Deep residual networks for image recognition]

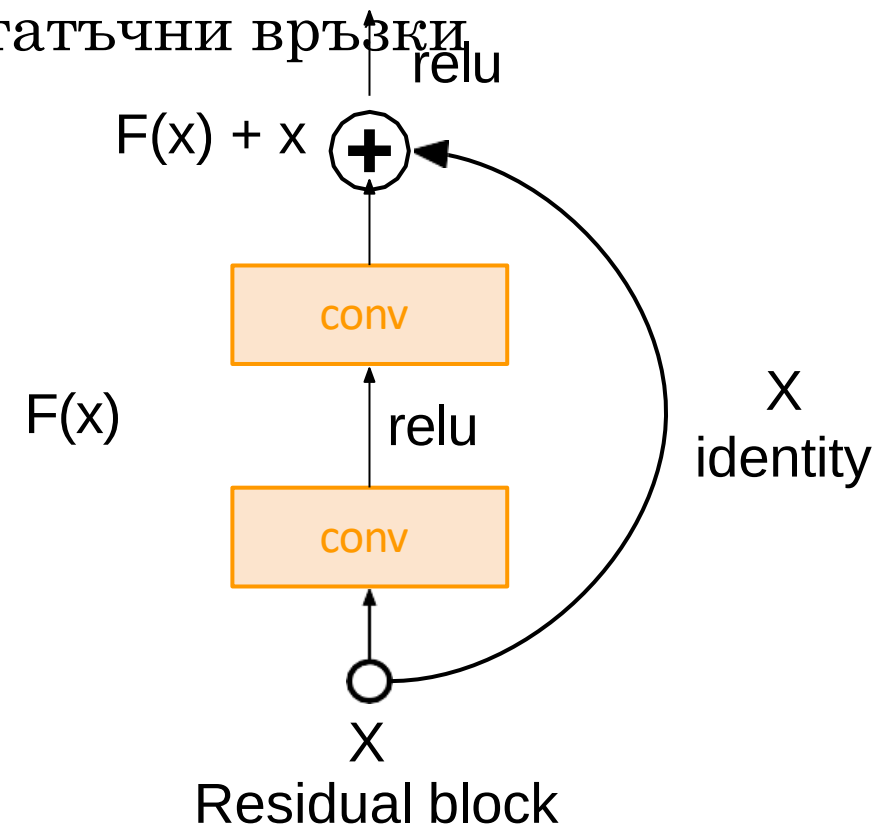
Residual Network



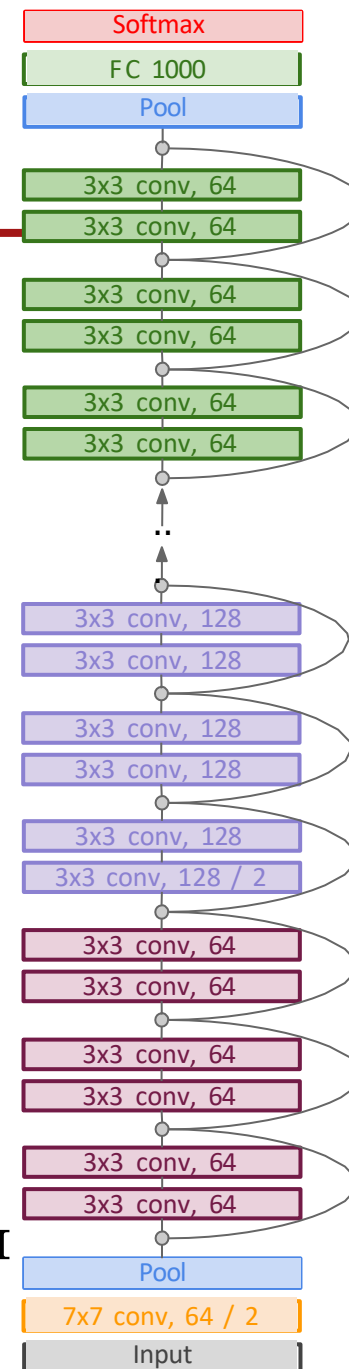
[He et al., 2015. Deep residual networks for image recognition]

ResNet

- Много дълбоки мрежи, използващи остатъчни връзки



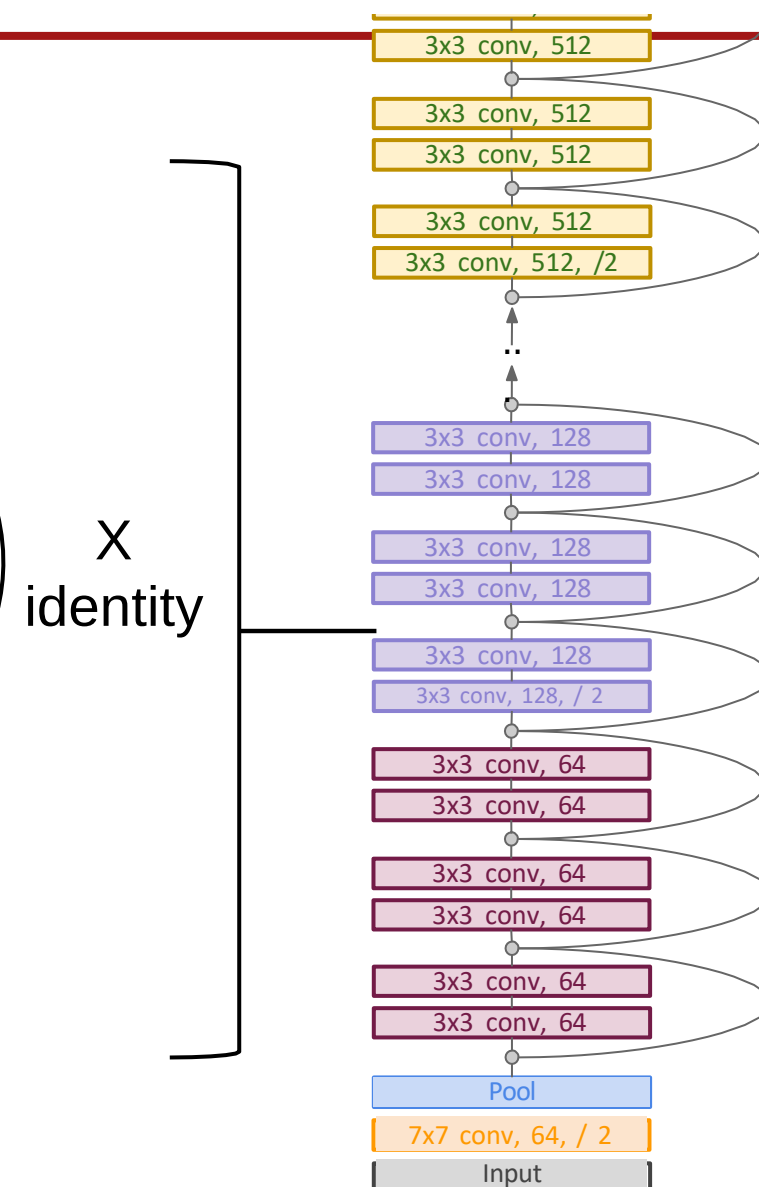
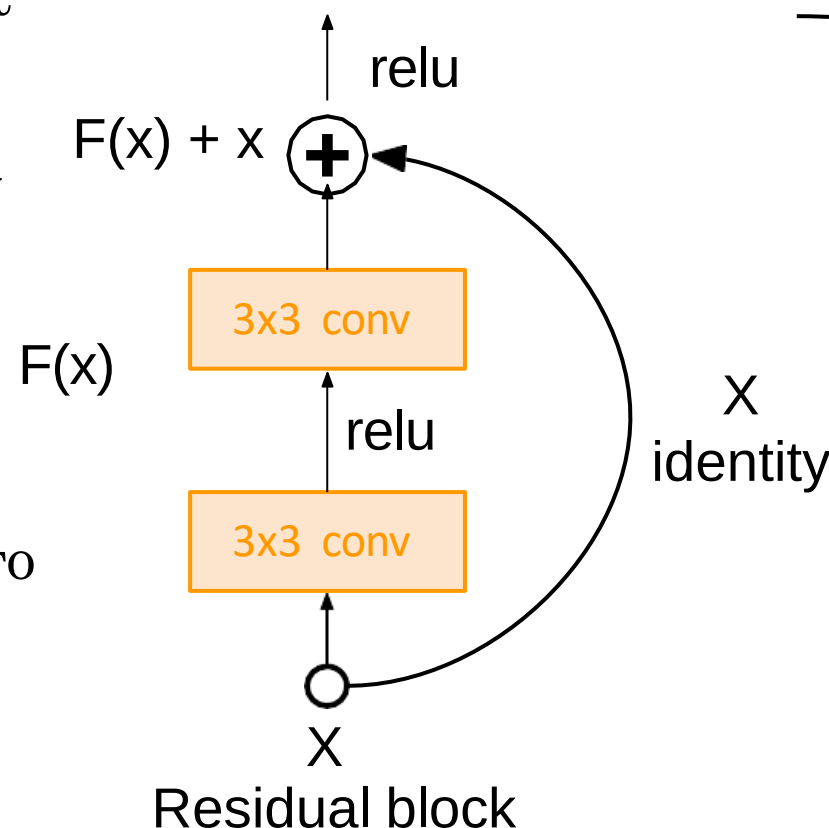
- 152-слоен модел за ImageNet
- Победител в класирането ILSVRC'15 (3.57% грешка в топ 5)
- Спечели всички състезания по класификация и откриване в ILSVRC'15 и COCO'15!



[He et al., 2015]

ResNet

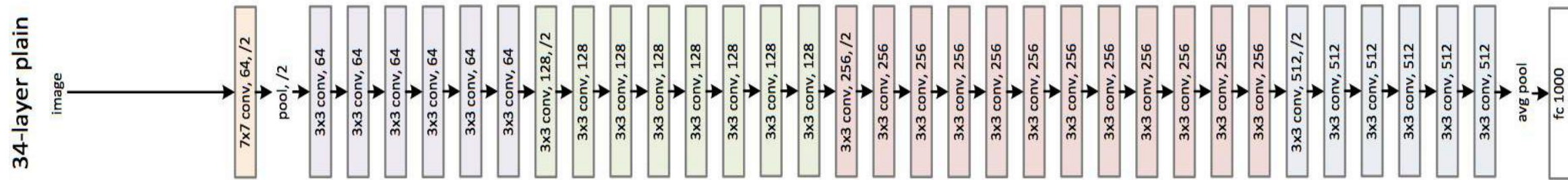
- Пълна архитектура на ResNet
- Остатъчни блокове на стека
- Всеки остатъчен блок има два 3x3 конв слоя
- Периодично удвоява # на филтрите и понижава пространствено със стъпка 2
- Допълнителен слой в началото
- Няма FC слоеве в края (само FC 1000 за изходни класове)



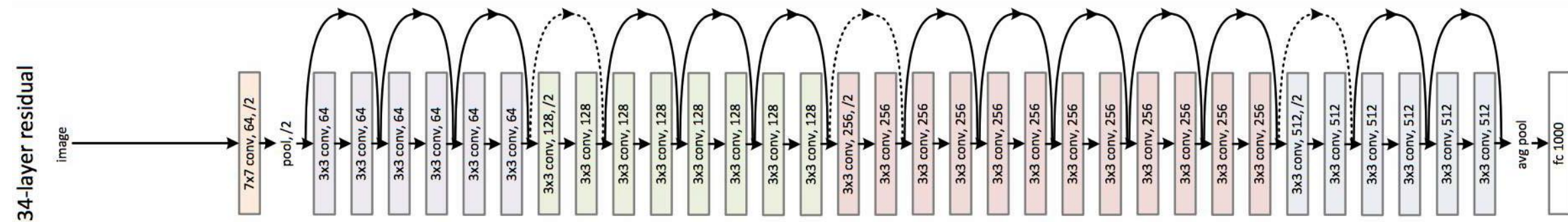
[He et al., 2015]

ResNet

Plain



ResNet



[He et al., 2015. Deep residual networks for image recognition]

MobileNet



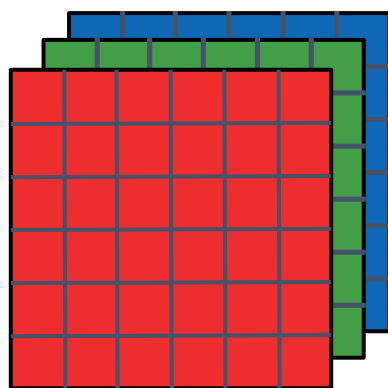
MobileNets

- Ниски изчислителни разходи при внедряване
- Полезно за мобилни и вградени приложения за зрение
- Нормални срещу разделени по дълбочина извивки



[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications]

Нормална конволюция



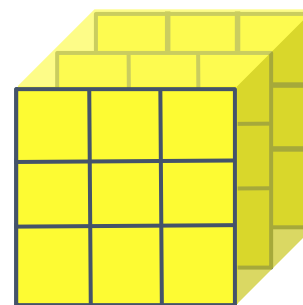
$$6 \times 6 \times 3$$

$$n \times n \times n_c$$

Computational cost

$$2160 = 27 \times 16 \times 5$$

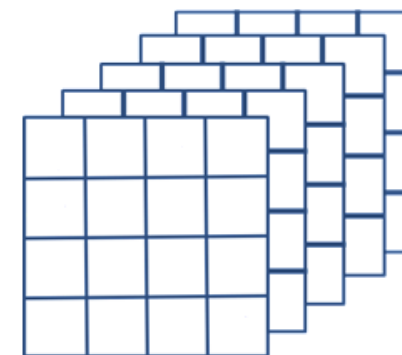
*



$$3 \times 3 \times 3$$

$$f \times f \times n_c$$

=



$$4 \times 4 \times 5$$

=

#filter params

$$3 \times 3 \times 3$$

x

filter positions

$$4 \times 4$$

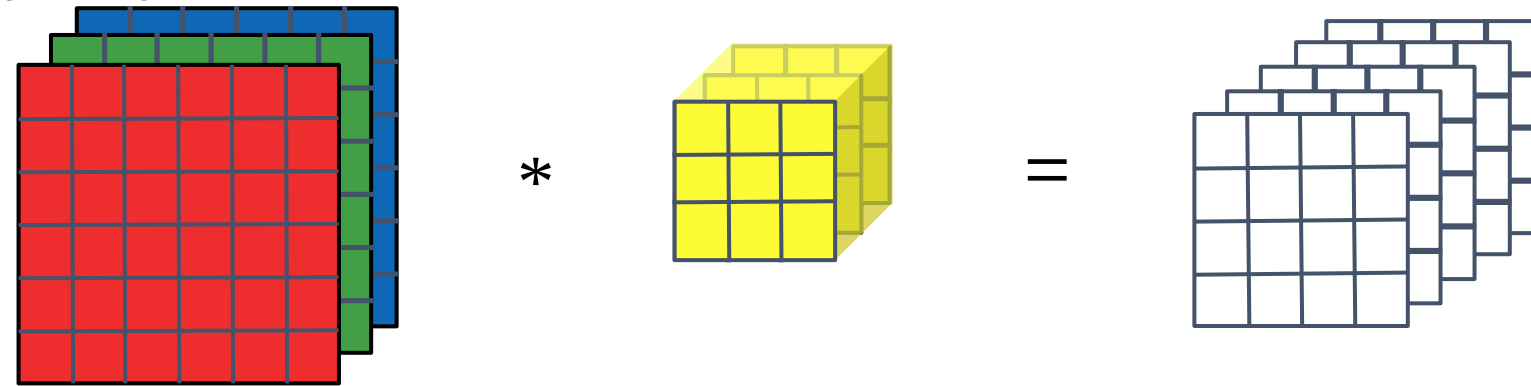
x

of filters

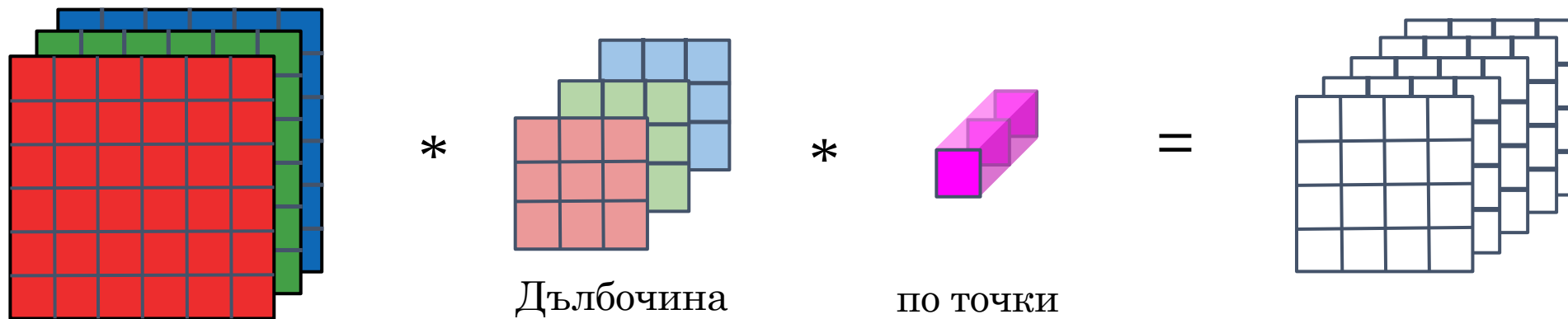
$$5$$

Дълбочинна сепарабилна конволюция

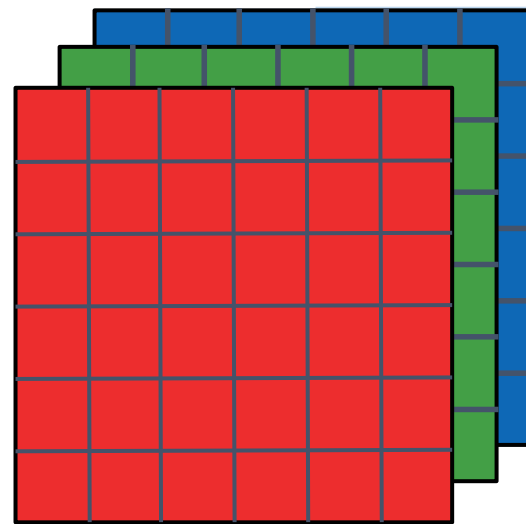
- Normal Convolution



- Depthwise Separable Convolution

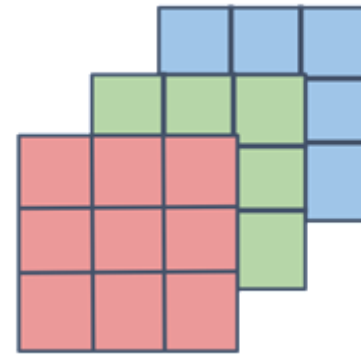


Дълбочинна конволюция



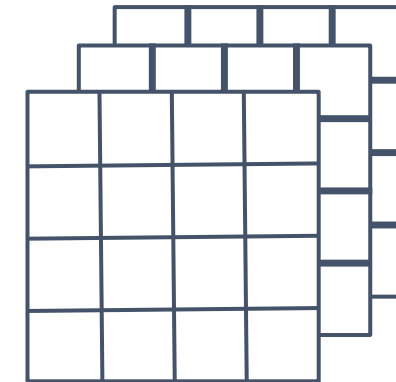
6 x 6 x 3

*



3 x 3

=



4 x 4 x 3

Computational cost

432

=

#filter params

x

filter positions

x

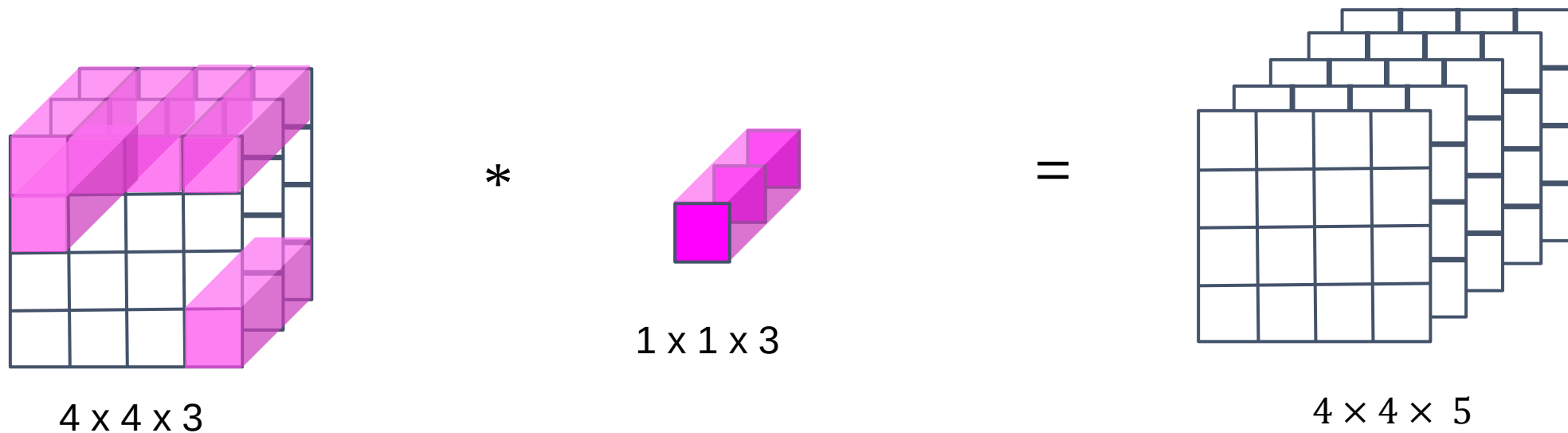
of filters

3 x 3

4 x 4

3

Точкова конволюция



Computational cost = #filter params x # filter positions x # of filters

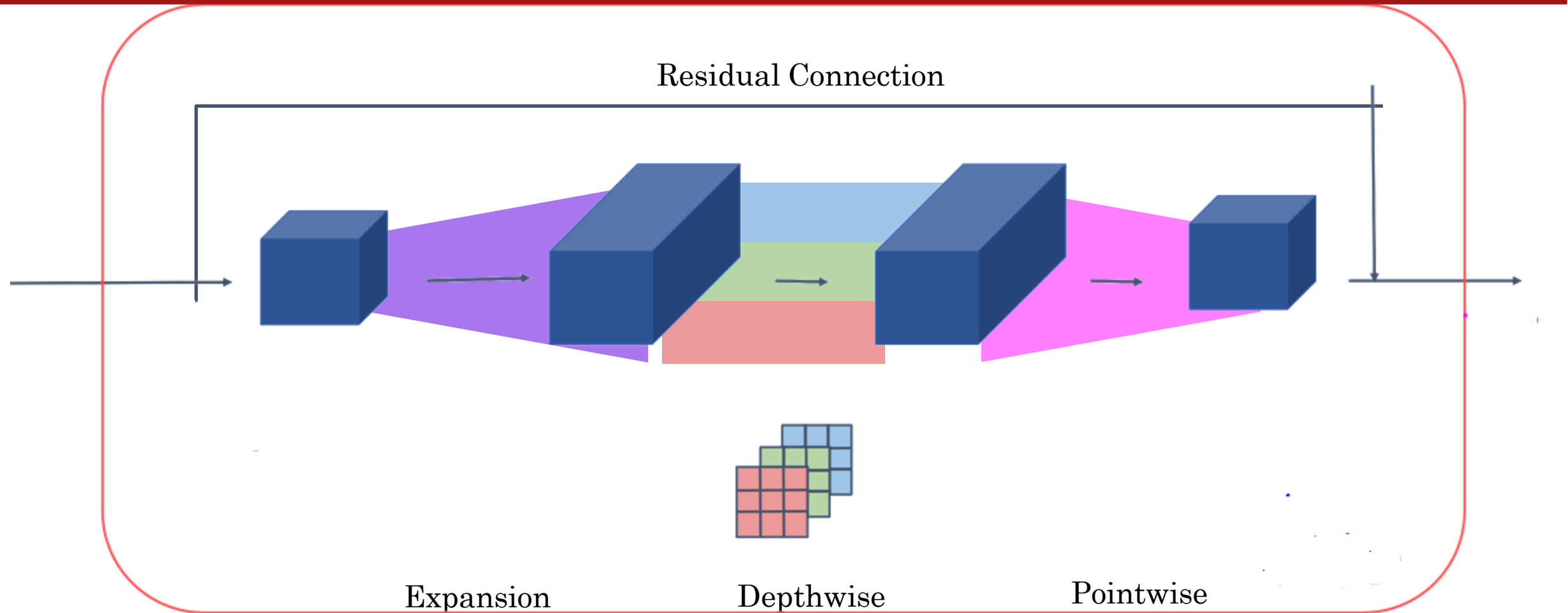
$$240 = 1 \times 1 \times 1 \times 4 \times 4 \times 5$$

Сравнение на операциите

- Цената на нормална конволюция в примера е 2160
- Цената на дълбочинно сепарабелна конволюция е $432+240= 672$

[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications]

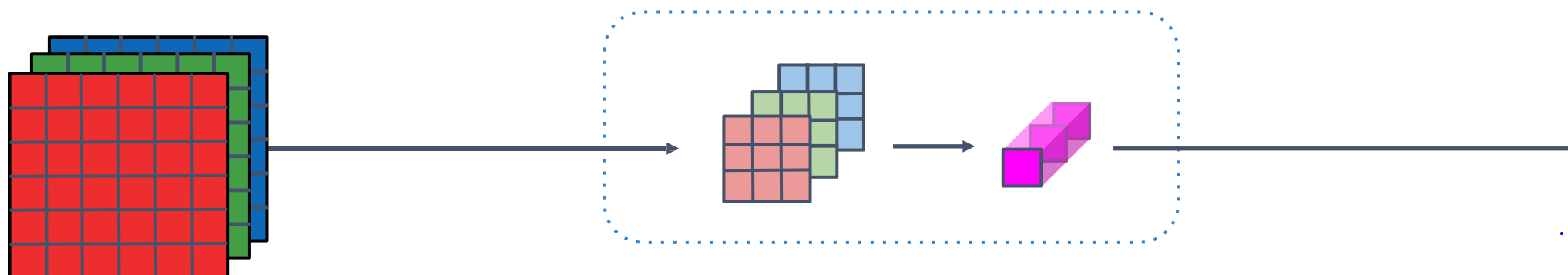
MobileNet v2 Bottleneck



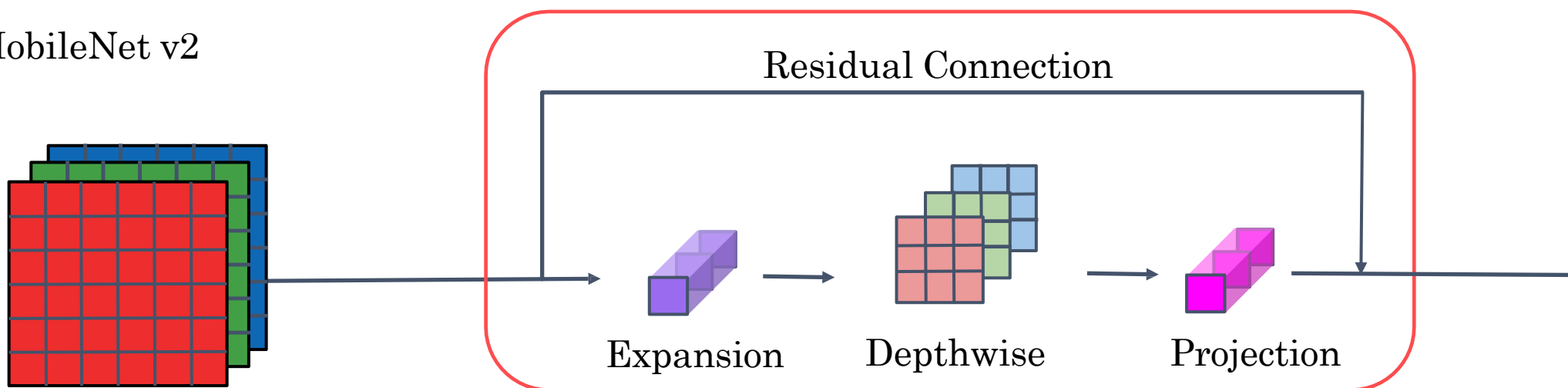
[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

MobileNet

MobileNet v1



MobileNet v2

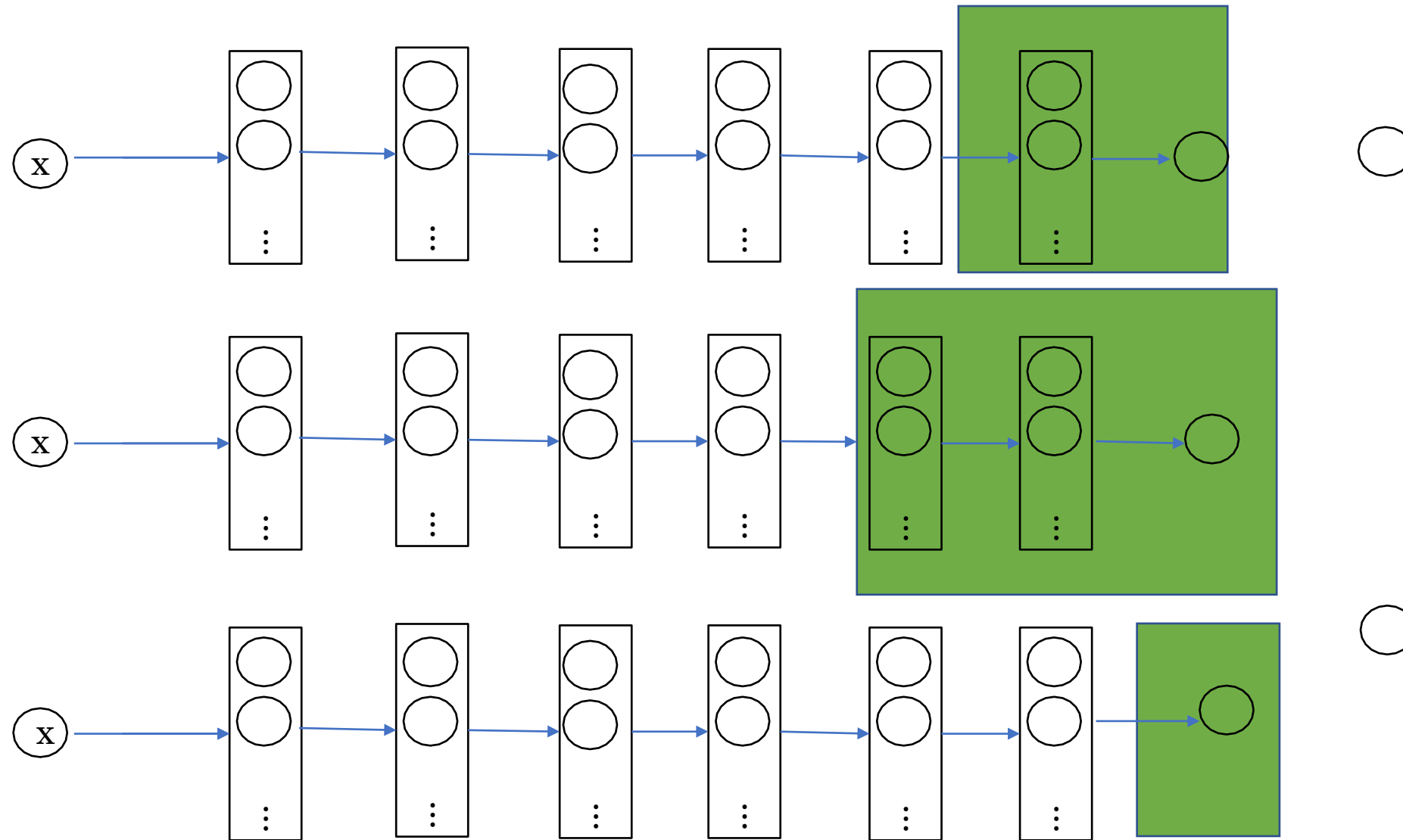


[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

Използване на ConvNets

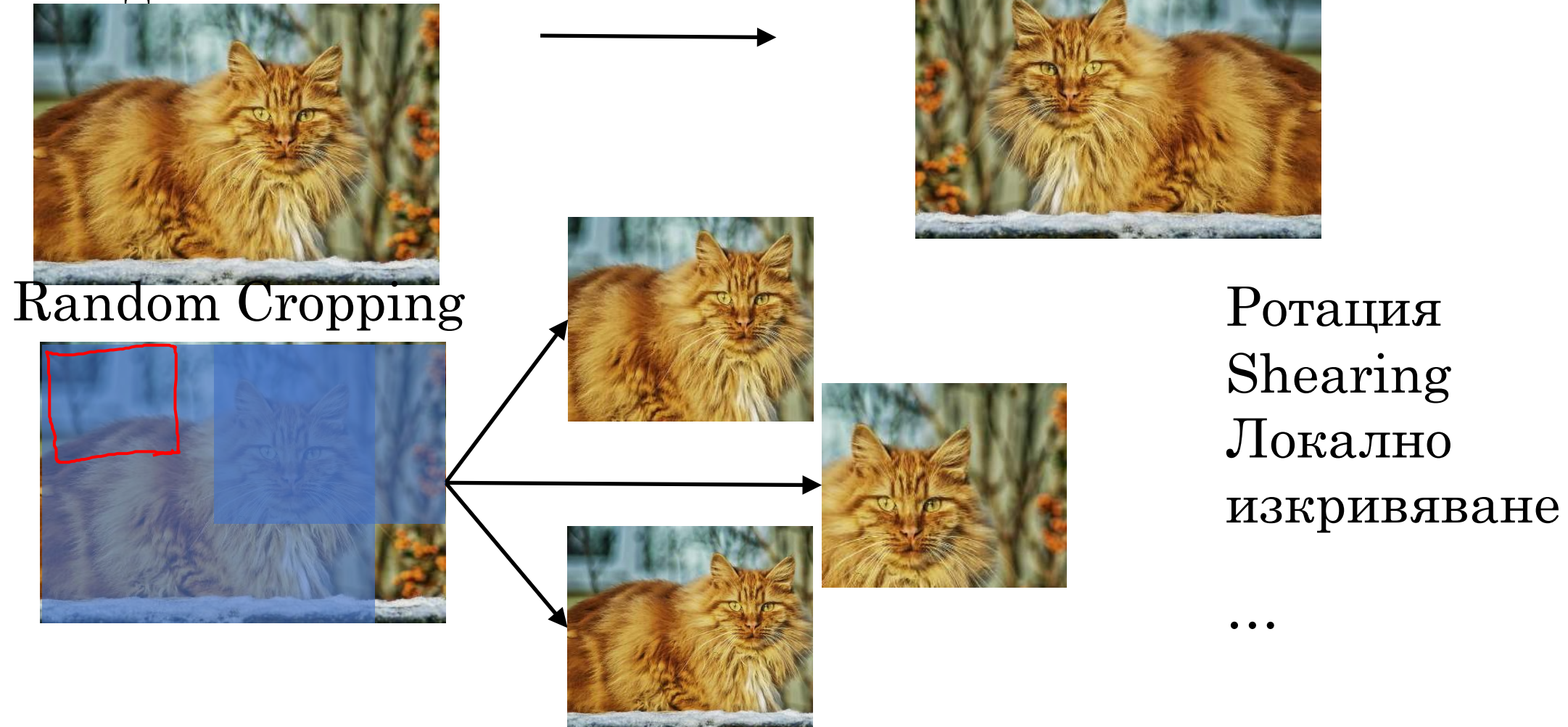
Трансферно обучение

Transfer Learning

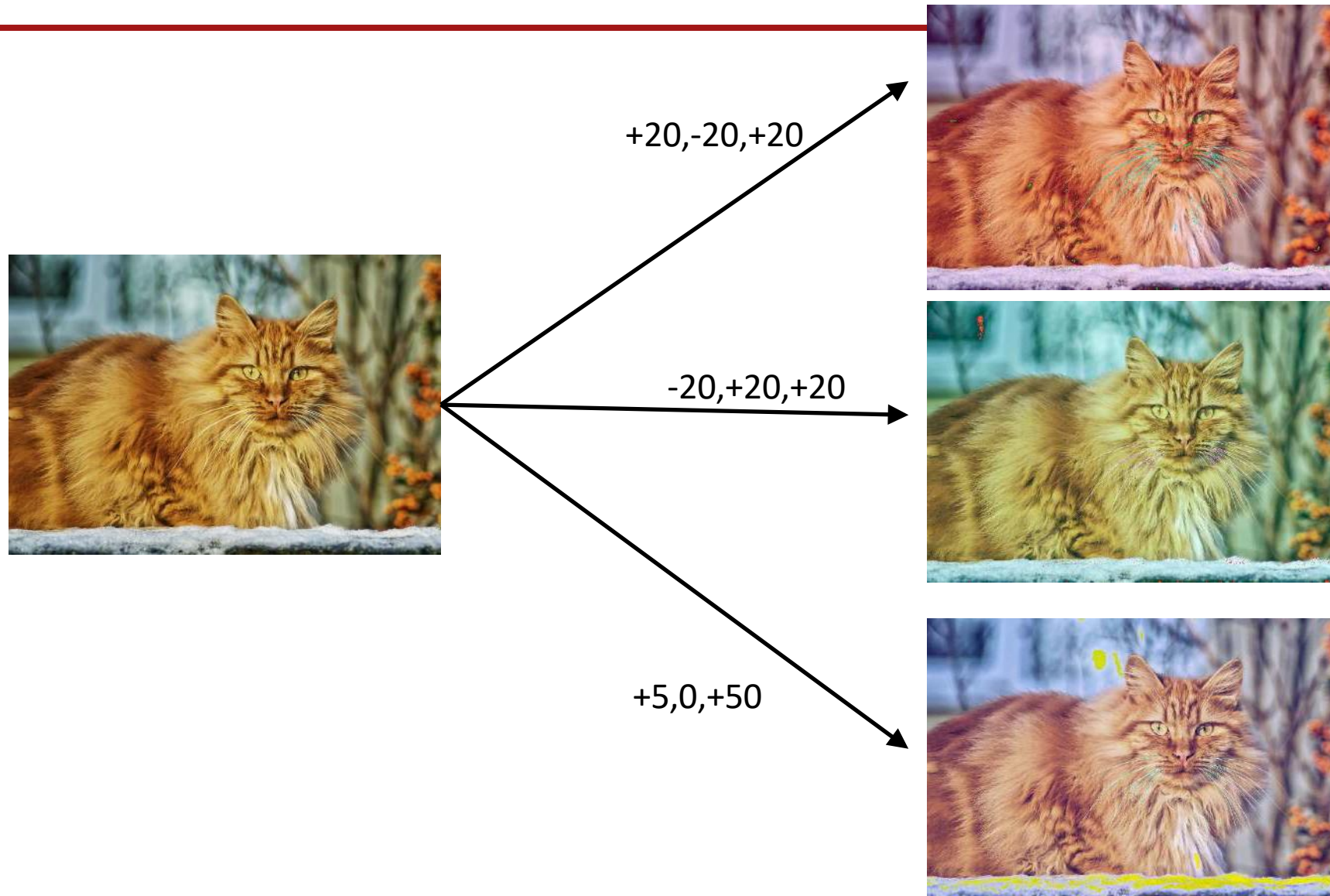


Общ метод за увеличаване

- Огледало



Промяна на цвета



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

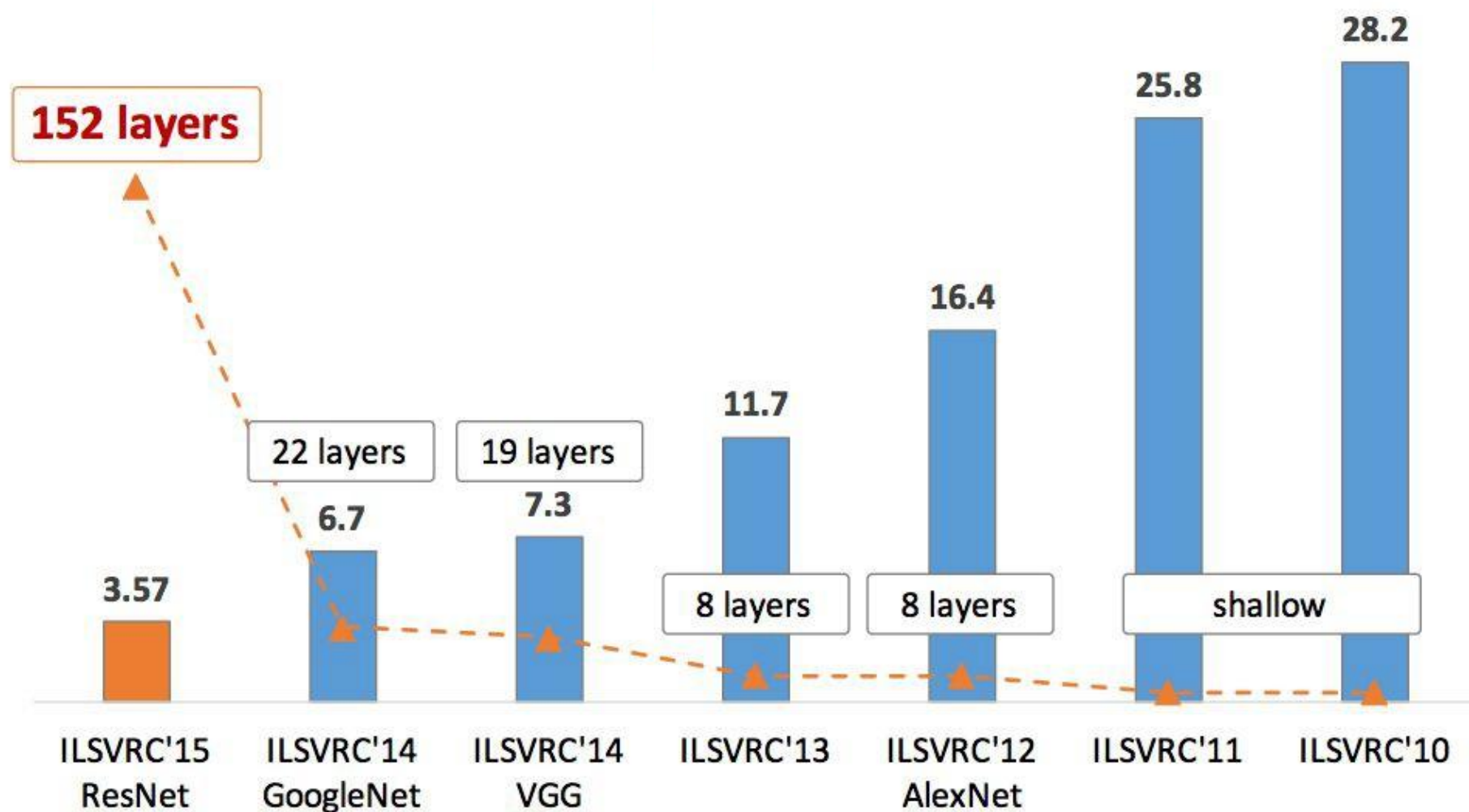
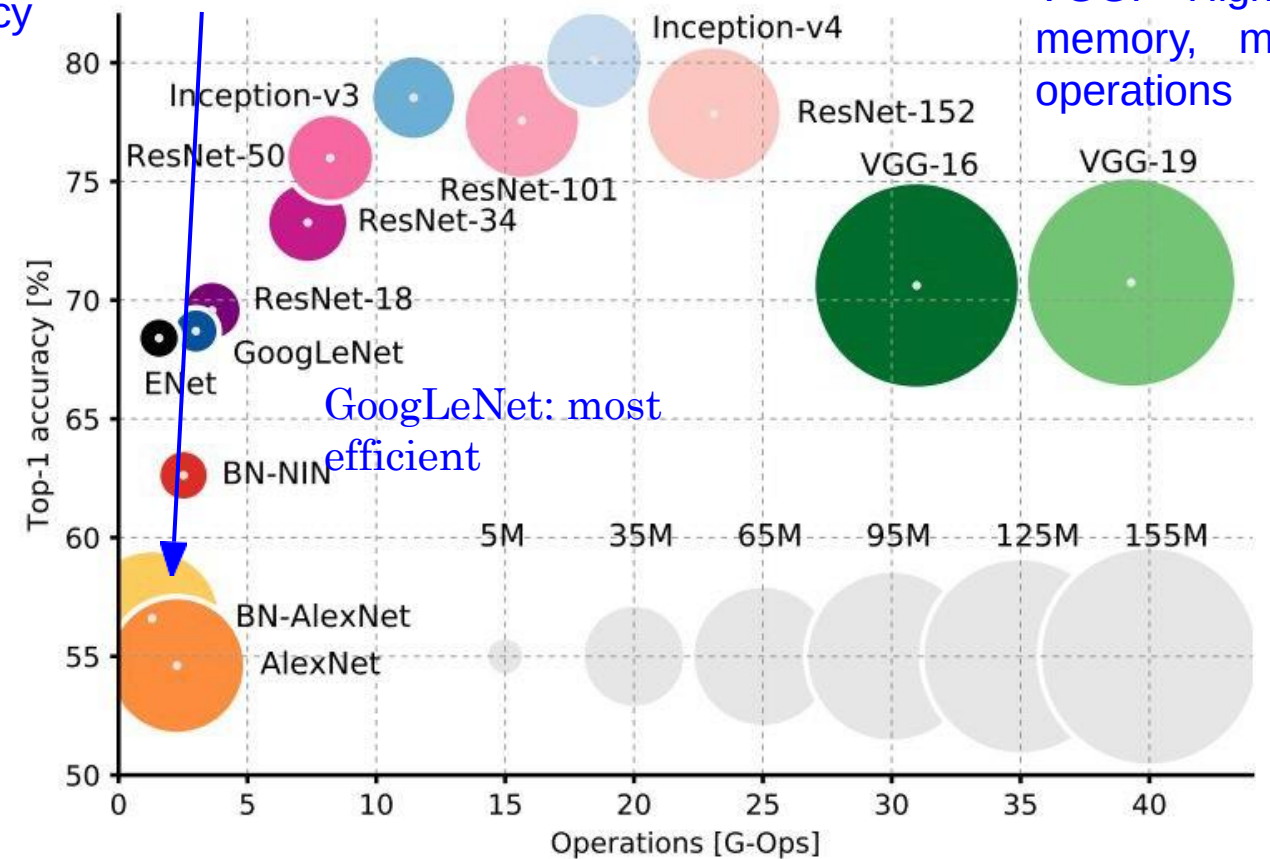
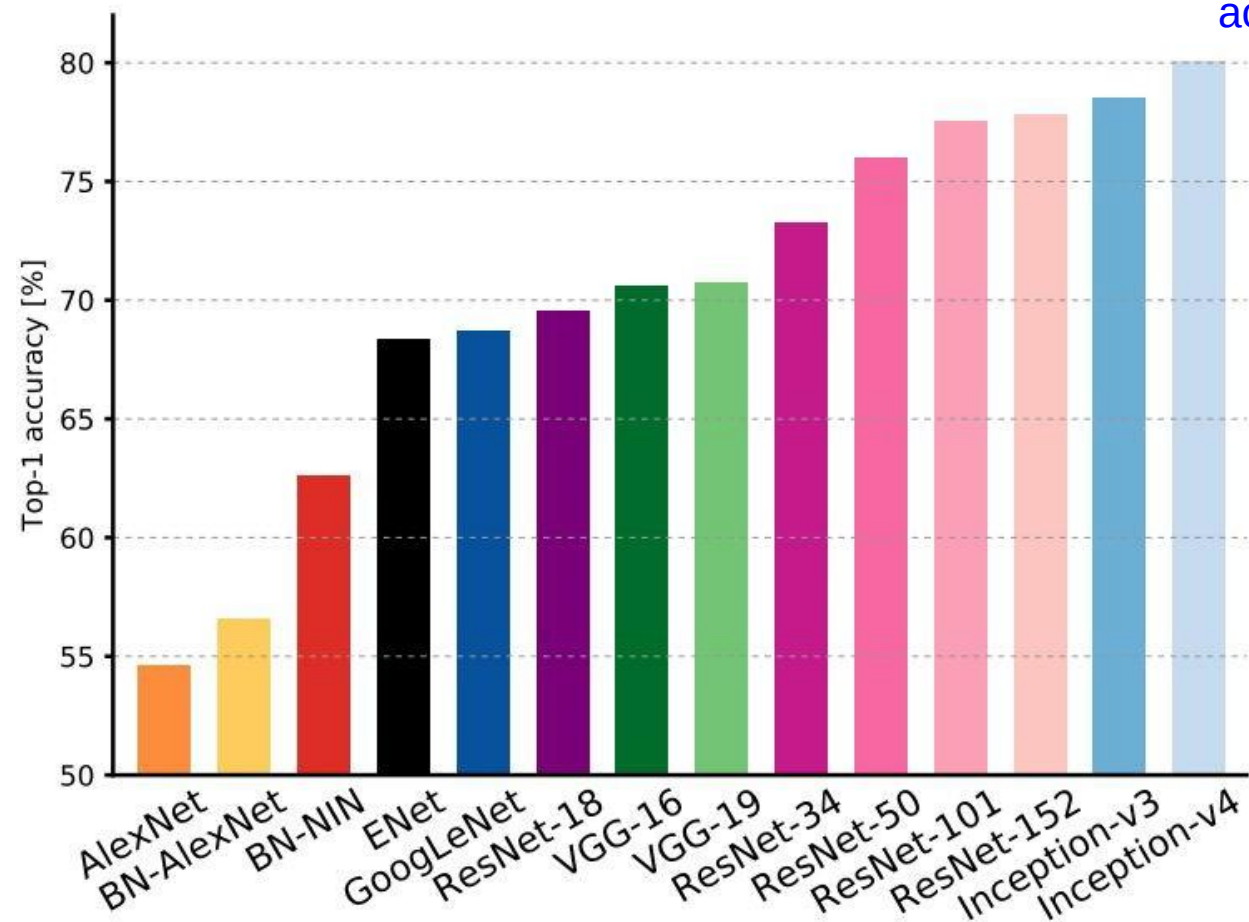


Figure copyright Kaiming He, 2016. Reproduced with permission.

Comparing complexity...

AlexNet:
Smaller compute, still
memory heavy, lower
accuracy

VGG: Highest
memory, most
operations



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Figures copyright Alfredo Canziani, Adam Paszke, Eugenio Culurciello, 2017. Reproduced with permission.